

System Specification

Document Metadata

Field	Value
Project Name	Local AI Task Automation System
Document Title	System Specification
Phase	Phase 1 — Requirement & System Specification
Version	1.0
Status	Draft — Pending User Approval
Prepared Date	2026-09-29
Intended Audience	System Owner, AI-assisted Developer, Developer/Maintainer, QA/Test Operator
Source Document	User-provided Phase 1 specification brief and the established project planning baseline
Document Owner	System Owner
Approval Status	Pending approval of documented open decisions
Deployment Model	Local-first, single Windows computer
Mandatory Cloud Dependency	None
Primary Backend	Python + FastAPI
Primary Database	SQLite
Browser Automation	Playwright

Desktop Automation	Windows UI Automation and/or pywinauto
Scheduling	Application scheduler + Windows Task Scheduler
Initial Frontend	HTML, CSS, JavaScript
Change-Control Rule	Changes to confirmed requirements, architecture, state model, database contracts, or safety-critical behavior MUST be recorded in the Decision Register and Requirements Traceability Matrix before implementation changes are accepted.

This document defines the implementation-ready requirements for the local Windows automation system. The attached Phase 1 brief explicitly establishes the local-first architecture, supported targets, task types, scheduling modes, lifecycle expectations, and the requirement that Phase 1 act as the single source of truth for later development.

1. Executive Summary

The Local AI Task Automation System is a local-first Windows application designed to allow one user to create, schedule, execute, monitor, retry, and review tasks that interact with authenticated AI application targets.

The primary problem being solved is repetitive manual intervention in long-running AI workflows. The user currently needs to return to a computer at predetermined intervals, open an appropriate AI session, enter either a continuation command or a new prompt with associated files, wait for the target application to respond, and repeat the process later. The proposed system automates those repetitive interactions while keeping execution on the user's own Windows computer.

The system **MUST** operate without Oracle Cloud, VPS infrastructure, mandatory cloud databases, or a cloud-hosted backend. The complete application stack runs locally on the Windows computer.

The system supports two initial task types:

1. **Prompt Task** — sends a user-defined prompt, optionally with local files, to a selected target conversation at a scheduled time.
2. **Continue Task** — sends the exact default continuation instruction Continue to a selected target conversation at a scheduled time or according to an interval.
- 3.

The initial execution targets are:

1. Google Chrome
2. Brave Browser
3. Microsoft Edge
4. Claude Desktop

Browser targets use persistent automation profiles that are deliberately separated from the user's normal personal browser profiles. The user performs the initial authentication manually. The automation layer then uses the persistent automation profile for subsequent execution.

Files remain on the user's local drive. The database stores only file paths and metadata; it **MUST NOT** store the binary contents of user files in the initial release. When execution begins, the local Python agent validates the stored path, accesses the file directly from the local filesystem, and attaches it to the selected AI target when the target supports attachment.

The application contains a local Web UI, local API/backend, local SQLite database, scheduler, queue engine, automation agent, target adapters, file-management component, logging/monitoring components, and Windows Task Scheduler integration.

The scheduler persists tasks in the database so that task definitions survive application restart, Windows restart, or sleep/wake transitions. Windows Task Scheduler is used to wake the computer when necessary. Once Windows is awake and the appropriate interactive user session is available, the local agent retrieves due work from the queue and executes it.

The system **MUST NOT** force sleep while an automation task is running, a protected operation is active, a retry is immediately pending, or recent user activity violates the configured automatic-sleep policy.

Successful completion of the project means that the user can create a task through the local Web UI, put the Windows computer into sleep mode, allow the scheduled wake-up to occur, have the correct target and conversation selected automatically, execute the task, record the result, calculate the next run when applicable, and return the computer to sleep when safe to do so.

2. Project Vision and Objectives

2.1 Vision Statement

Provide a dependable, local-only automation system that transforms repetitive scheduled interaction with authenticated AI sessions into a persistent, recoverable, user-controlled workflow.

2.2 Primary Objectives

The system MUST:

- automate Prompt and Continue tasks;
- support Chrome, Brave, Edge, and Claude Desktop;
- maintain persistent task definitions locally;
- retain only local file references and metadata rather than binary file content;
- schedule one-time and recurring work;
- support every-N-hours execution;
- wake Windows when required;
- prevent duplicate or unsafe execution where outcome is uncertain;
- detect and record execution failures;
- provide controlled retry and recovery;
- preserve execution history;
- expose system health through a local Web UI;
- allow manual intervention when automation cannot safely continue;
- remain usable without a cloud backend.

2.3 Measurable Success Outcomes

The system is considered functionally successful when all of the following can be demonstrated on the real Windows computer:

1. A Prompt Task can be created from the Web UI.
2. A Continue Task can be created from the Web UI.
3. Tasks persist after application restart.

4. Target selection works for all supported targets after target-specific setup.
5. Local files can be referenced by path and attached during execution.
6. Scheduled tasks execute without the UI remaining open.
7. A sleeping Windows computer can wake and process a valid scheduled task under supported power and session conditions.
8. Execution status and evidence are recorded.
9. Retryable failures are retried according to policy.
10. Uncertain send outcomes are not blindly duplicated.
11. The application can return the computer to sleep when the configured conditions are satisfied.
12. Database and logs remain recoverable after normal application or Windows restarts.

2.4 User Value

The system removes repetitive physical interaction while retaining human control over task creation, target selection, schedule definition, file selection, cancellation, review, and exception handling.

2.5 System Boundaries

The system controls only the local Windows environment and the supported AI applications through their observable user interfaces.

It does not become a replacement for the AI application, does not provide an alternative API for the target service, and does not bypass target-service authentication, access controls, CAPTCHA, rate limits, or other platform safeguards.

2.6 Phase 1 Objectives

Phase 1 establishes:

- complete system scope;
- functional requirements;
- non-functional requirements;
- architecture;
- data model;

- task lifecycle;
- scheduling semantics;
- queue behavior;
- target model;
- file rules;
- error and recovery behavior;
- security rules;
- test strategy;
- acceptance criteria;
- implementation phase mapping.

2.7 Relationship to Subsequent Phases

Phase 1 defines the contract that later development phases implement. Later phases MAY refine implementation details but MUST NOT silently redefine confirmed behavior.

3. Scope

3.1 In Scope

The initial release includes:

- local Windows deployment;
- local Web UI;
- local FastAPI backend;
- SQLite database;
- task CRUD and lifecycle management;
- Prompt Tasks;
- Continue Tasks;
- Chrome target;
- Brave target;

- Edge target;
- Claude Desktop target;
- persistent automation profiles for browser targets;
- manually authenticated user-controlled sessions;
- local file path storage;
- file existence and accessibility validation;
- file attachment workflows;
- one-time scheduling;
- recurring scheduling;
- every-N-hours scheduling;
- queue management;
- single-worker default execution;
- execution history;
- logging;
- retry/recovery;
- Windows Task Scheduler integration;
- wake/sleep coordination;
- dashboard;
- configuration;
- packaging;
- local backup and restore.

3.2 Out of Scope

The initial release does not include:

- mandatory cloud hosting;
- Oracle Cloud;
- mandatory VPS;

- cloud database;
- remote multi-user access;
- Internet-accessible public API;
- multi-user enterprise permissions;
- bypassing CAPTCHA;
- bypassing authentication;
- credential harvesting;
- extracting secrets from browser profiles;
- rate-limit evasion;
- stealth automation;
- parallel execution of the same target profile;
- automated unlocking of the Windows login screen;
- guaranteed GUI automation while the Windows session is locked;
- automatic recovery from every possible target-service UI redesign;
- storing full AI response bodies by default;
- binary file storage inside SQLite.

3.3 Future Scope

Potential future extensions include:

- React frontend migration;
- additional AI target adapters;
- conditional workflows;
- task dependencies beyond basic prerequisite relationships;
- workflow templates;
- advanced result extraction;
- optional full response archiving;
- multiple local Windows agents;

- LAN-controlled administration;
- optional remote synchronization;
- advanced visual locator recovery;
- task-level notifications;
- more sophisticated workflow orchestration.

These are not required for Phase 1 completion.

3.4 Constraints

Local-only Deployment

All required application services **MUST** run on the Windows computer.

Windows Dependency

The system depends on Windows-specific facilities for power management and desktop automation.

Supported Targets

Only the four initial targets are guaranteed within this specification.

User-Authenticated Sessions

The system assumes that the user has legitimately authenticated the target application and that authentication remains valid.

Internet Dependency

The local application does not require the Internet for task storage, scheduling, or UI operation, but the target AI applications generally require Internet/network availability for remote service interaction.

UI Automation Fragility

Browser and desktop automation depend on target UI behavior. Application UI updates may break locators or execution sequences.

Sleep/Wake Limitations

Wake and sleep behavior depends on Windows configuration, power policy, BIOS/UEFI, hardware, drivers, battery state, user session state, and other environmental conditions.

External Service Changes

Target-service UI changes, session policies, upload rules, or other external changes can affect automation.

File Availability

A file path remains usable only while the file exists and remains accessible.

Application Updates

Browser and desktop-app updates can alter executable paths, UI structure, permissions, or automation behavior.

Browser Profile Dependencies

Target execution depends on configured automation profiles and their session state.

3.5 Dependencies

- Windows 10/11 environment with the required power-management capabilities;
- Python runtime supported by the pinned project dependency set;
- Playwright and supported browsers;
- target applications installed;
- user-authenticated target sessions;
- local filesystem access;
- Windows Task Scheduler;
- sufficient user permissions;
- valid database storage path;
- target service availability.

4. Stakeholders and User Roles

4.1 System Owner

Responsibilities

- defines tasks;
- selects target and conversation;
- selects files;

- controls schedules;
- reviews status;
- approves configuration;
- resolves manual-intervention conditions.

Permissions

Full local application control.

Prohibited Actions

The system does not permit the owner to use the automation layer to bypass external authentication or access controls.

Interaction

Uses the Web UI and occasionally the Windows environment for setup and recovery.

4.2 Local Administrator

The Local Administrator is the Windows account or operator capable of installing dependencies and configuring Windows services.

Responsibilities

- install software;
- configure Windows Task Scheduler;
- configure power settings;
- manage permissions;
- install browser/desktop applications;
- maintain the application environment.

Permissions

Operating-system-level permissions as granted by Windows.

Prohibited Actions

No special application-level bypass capability is created by this role.

4.3 Operator/User

In the initial release, this role is typically the same person as the System Owner.

Responsibilities include:

- creating tasks;
- monitoring tasks;
- handling alerts;
- resolving failed target configuration;
- manually re-authenticating targets when required.

4.4 Developer/Maintainer

Responsibilities

- maintain source code;
- update dependencies;
- repair adapters;
- maintain database migrations;
- extend target capabilities;
- maintain tests.

Permissions

Source-code and maintenance-level access.

Prohibited Actions

Must not hard-code or extract user authentication secrets.

4.5 Scheduler/Agent Process

This is a technical actor rather than a human role.

Responsibilities:

- identify due tasks;
- claim queue entries;
- invoke target adapters;
- collect evidence;
- record execution state;

- calculate future runs;
 - coordinate wake/sleep behavior.
-

5. User Personas and Primary Use Cases

UC-001 — Create One-Time Prompt Task

Actor

System Owner

Preconditions

- Web UI is running;
- selected target is enabled;
- conversation is configured;
- prompt is non-empty.

Trigger

User opens Create Prompt Task.

Main Flow

1. User selects target.
2. User selects or configures conversation.
3. User enters prompt.
4. User optionally selects local files.
5. User selects one-time schedule.
6. System validates task.
7. System stores task.
8. System schedules execution.
9. Task state becomes SCHEDULED.

Alternate Flow

User selects immediate/manual execution instead of scheduled execution.

Exceptions

- target disabled;
- conversation invalid;
- file missing;
- schedule invalid.

Postconditions

Task is stored and ready for execution.

Evidence

Task record, validation events, schedule information.

UC-002 — Create Recurring Continue Task**Actor**

System Owner

Preconditions

- target and conversation are valid.

Trigger

User selects Continue Task.

Main Flow

1. User selects target.
2. User selects conversation.
3. System uses the default continuation text Continue.
4. User specifies initial execution time.
5. User specifies recurrence.
6. System validates.
7. Task is stored.
8. First run is scheduled.

Exceptions

Invalid interval, disabled target, missing conversation.

Postconditions

Recurring task enters SCHEDULED.

Evidence

Task definition, schedule, creation event.

UC-003 — Execute Scheduled Prompt with Files**Actor**

Scheduler/Agent

Preconditions

- task is due;
- target is available;
- file paths remain valid;
- interactive user session is usable.

Trigger

Scheduler identifies a due task.

Main Flow

1. Claim queue entry.
2. Validate target.
3. Validate conversation.
4. Validate files.
5. Open target.
6. Navigate to conversation.
7. Attach files.
8. Send prompt.

9. Wait for observable completion.
10. Record result.
11. Calculate next run.
12. Release target.
13. Evaluate sleep policy.

Exceptions

- file missing;
- browser unavailable;
- conversation not found;
- upload failure;
- response timeout.

Postconditions

Successful attempt is recorded or task enters the appropriate recovery state.

Evidence

Execution run, attempt record, events, safe screenshots where configured.

UC-004 — Execute Continue Task

Actor

Scheduler/Agent

Preconditions

- task due;
- target available;
- conversation available.

Main Flow

1. Claim queue entry.
2. Validate target.

3. Open conversation.
4. Send exact text Continue.
5. Wait for completion signal.
6. Record result.
7. Calculate next run.
8. Release target.
9. Evaluate sleep policy.

Exceptions

Same classes as Prompt Task except file-related steps are absent.

Postconditions

Execution is recorded.

Evidence

Execution event log and task-run result.

UC-005 — Execute While Computer Is Sleeping**Actor**

Windows Task Scheduler + Agent

Preconditions

- task exists;
- wake integration has been configured;
- Windows permits wake timer behavior;
- hardware/power configuration permits wake.

Trigger

Task wake time arrives.

Main Flow

1. Windows wakes if the platform permits.

2. Required local components start or resume.
3. Scheduler discovers due task.
4. Agent claims it.
5. Task executes.

Alternate Flow

Computer wakes late.

System identifies overdue work and applies missed-run policy.

Exceptions

Computer does not wake, user session is unavailable, or target application cannot run in the current session.

Postconditions

Task executes if conditions permit; otherwise a recoverable failure or manual-action state is recorded.

Evidence

Power-event log, scheduler event, execution state.

UC-006 — Retry a Failed Task

Actor

Scheduler/Agent

Trigger

A retryable error is detected.

Main Flow

1. Error classified.
2. Retry eligibility evaluated.
3. Retry delay calculated.
4. Task enters RETRY_SCHEDULED.
5. New queue entry is created.

6. Retry executes.
7. Result is recorded.

Exception

If completion status is uncertain after a send, system **MUST NOT** automatically resend without a safe verification method.

Postcondition

Task either succeeds or enters FAILED / MANUAL_ACTION_REQUIRED.

UC-007 — Resolve Missing File**Actor**

System Owner

Trigger

Execution detects missing or inaccessible file.

Main Flow

1. Task enters MANUAL_ACTION_REQUIRED.
2. UI identifies the missing file path.
3. User updates the file reference or restores the file.
4. User validates task.
5. User manually retries or resumes task.

Postcondition

Task becomes executable again or is cancelled.

UC-008 — Resolve Expired Target Session**Actor**

System Owner

Trigger

Target health check or execution identifies missing authentication/session state.

Main Flow

1. Task becomes MANUAL_ACTION_REQUIRED.
2. UI displays the affected target.
3. User re-authenticates manually in the configured automation profile.
4. Target health check is rerun.
5. User resumes or retries the task.

The system **MUST NOT** collect or store the user's password.

6. Functional Requirements

Normative terms:

- **MUST** — mandatory requirement.
- **MUST NOT** — prohibited behavior.
- **SHOULD** — recommended behavior.
- **MAY** — optional behavior.

6.1 Task Management

FR-001 — Task Creation

The system **MUST** allow creation of a task from the local Web UI.

FR-002 — Task Viewing

The system **MUST** allow the user to view task definitions and current states.

FR-003 — Task Editing

The system **MUST** allow editing of tasks that are not currently executing.

FR-004 — Task Duplication

The system **SHOULD** allow duplicating a task while assigning a new task ID.

FR-005 — Enable/Disable

The system **MUST** allow scheduled tasks to be enabled or disabled without deleting task definitions.

FR-006 — Pause/Resume

The system **MUST** allow eligible recurring tasks to be paused and resumed.

FR-007 — Cancellation

The system **MUST** allow queued or scheduled tasks to be cancelled.

FR-008 — Archive/Delete

The system **MUST** provide safe archival and **MAY** provide permanent deletion subject to configured policy.

FR-009 — Manual Run

The system **MUST** allow an eligible task to be manually executed immediately.

Manual execution **MUST NOT** silently alter its normal schedule unless explicitly configured.

FR-010 — Validation and History

The system **MUST** validate tasks before scheduling and **MUST** preserve execution history separately from the reusable task definition.

6.2 Task Types

FR-011 — Prompt Task

A Prompt Task **MUST** contain:

- task name;
- target;
- conversation reference;
- non-empty prompt text;
- schedule;
- optional file references.

FR-012 — Continue Task

A Continue Task **MUST** send the exact default text:

Continue

Custom continuation text is NOT part of the initial release.

A future release MAY add custom continuation text as a separate feature.

FR-013 — Task-Type Isolation

A Continue Task MUST NOT accidentally send a Prompt Task's prompt or attached files.

A Prompt Task MUST NOT silently transform itself into a Continue Task.

6.3 Target Management

FR-014 — Target Registry

The system MUST maintain local target records for:

- Google Chrome;
- Brave Browser;
- Microsoft Edge;
- Claude Desktop.

FR-015 — Target Enable/Disable

Each target MUST have an enabled state.

FR-016 — Target Configuration

The system MUST store configurable executable and profile references where applicable.

FR-017 — Session Readiness

The system MUST support a target health/readiness check.

FR-018 — Capability Declaration

Each target MUST expose capability flags such as:

- prompt support;
- Continue support;
- file attachment support;
- conversation navigation;

- health check support.

FR-019 — Validation Evidence

The system SHOULD store the last successful target validation timestamp and result.

6.4 Conversation Management

FR-020 — Conversation Reference

Each task MUST refer to a specific target conversation.

FR-021 — Flexible Conversation Locator

The system MUST support a target-specific conversation locator strategy rather than assuming a universal stable conversation ID.

Supported logical reference types MAY include:

- direct URL;
- target-specific route;
- user-provided identifier;
- displayed title plus verification strategy.

FR-022 — Conversation Validation

Before execution, the target adapter MUST verify that the intended conversation is present or confidently identifiable.

FR-023 — Duplicate Name Handling

If multiple conversations have identical display names, the system MUST require a more specific locator.

FR-024 — Missing Conversation Handling

A missing or ambiguous conversation MUST produce `MANUAL_ACTION_REQUIRED` or another explicitly defined non-execution state.

FR-025 — Conversation Locking

Only one execution run SHOULD control a given conversation at a time.

6.5 File Material Management

FR-026 — Local File Selection

The UI **MUST** allow selecting local files through a native or browser-based file picker.

FR-027 — Path Storage

The system **MUST** store the absolute or normalized local file path.

FR-028 — Metadata Storage

The system **SHOULD** store:

- filename;
- size;
- modified time;
- hash where enabled;
- file type.

FR-029 — Multiple Files

A task **MAY** reference multiple local files.

FR-030 — File Ordering

The system **MUST** preserve the order of files as selected or explicitly reordered by the user.

FR-031 — File Pre-Validation

Before execution, all referenced files **MUST** be validated for existence and accessibility.

FR-032 — File Attachment

The local agent **MUST** access referenced files directly from the local filesystem and attach them to the selected target when supported.

FR-033 — Binary Storage Prohibition

The database **MUST NOT** store source-file binary content in the initial release.

FR-034 — File Change Detection

The system **SHOULD** detect when a referenced file has changed since task creation where metadata is available.

FR-035 — Missing File Handling

Missing or inaccessible files MUST prevent execution unless the task explicitly has no file requirement.

6.6 Scheduling

FR-036 — One-Time Scheduling

The system MUST support one-time execution.

FR-037 — Recurring Scheduling

The system MUST support recurring execution.

FR-038 — Every-N-Hours Scheduling

The system MUST support recurring schedules defined by a positive hour interval.

FR-039 — Time Zone

Schedules MUST be displayed using the Windows local time zone and persisted in UTC-compatible form.

FR-040 — Missed Run Handling

Missed one-time tasks SHOULD be executed as soon as possible unless expired.

Missed recurring occurrences SHOULD be skipped by default rather than executing a burst of obsolete runs.

FR-041 — Next-Run Calculation

The system MUST calculate and persist next_run_at.

FR-042 — Restart Persistence

Scheduling state MUST survive application and Windows restarts.

FR-043 — Fixed-Delay Every-N-Hours

The recommended default for Continue Tasks using every-N-hours scheduling is fixed-delay from the last successful completion.

FR-044 — Schedule Editing

Editing a schedule MUST recalculate future execution times without corrupting completed history.

FR-045 — Invalid Schedule Prevention

The system **MUST** reject non-positive intervals, invalid timestamps, and contradictory recurrence settings.

6.7 Queue Management

FR-046 — Queue Ordering

Queue selection **MUST** consider:

1. manually requested immediate runs;
2. due retry work;
3. normal due scheduled tasks;
4. priority;
5. due time;
6. creation order as a deterministic tie-breaker.

FR-047 — Single-Worker Default

The initial release **MUST** use a single execution worker.

This prevents collisions between GUI automation processes that share physical desktop resources and reduces risk of simultaneous control of browser or desktop sessions.

FR-048 — Task Claiming

Queue entries **MUST** be atomically claimable.

FR-049 — Duplicate Execution Prevention

A task run **MUST NOT** be simultaneously claimed by two execution workers.

FR-050 — Stale Claim Recovery

If a worker crashes, stale CLAIMED or RUNNING work **MUST** be recoverable using timestamps and recovery rules.

FR-051 — Queue Cancellation

A queued task **MUST** be cancellable before execution begins.

6.8 Browser Automation

FR-052 — Adapter Model

The scheduler **MUST** communicate with browser targets through a target-adapter abstraction.

FR-053 — Standard Adapter Operations

The adapter contract **MUST** logically support:

- `open_target();`
- `open_conversation();`
- `attach_files();`
- `send_prompt();`
- `send_continue();`
- `wait_for_response();`
- `detect_completion();`
- `detect_error();`
- `collect_execution_evidence();`
- `close_or_release_target();`

FR-054 — Persistent Profiles

Browser execution **MUST** use persistent automation profiles.

FR-055 — Personal Profile Separation

Automation profiles **MUST** be separate from the user's normal personal browser profile by default.

FR-056 — Session Use

Automation **MUST** use only legitimately authenticated user-controlled sessions.

FR-057 — Configurable Paths

Browser executable and profile paths **MUST** be configurable.

FR-058 — Failure Detection

The browser executor MUST detect browser launch failures, navigation failures, missing UI elements, and target timeouts.

FR-059 — Target UI Changes

The adapter MUST fail safely rather than blindly clicking unknown UI elements when locators no longer match expected behavior.

The automation MUST NOT bypass CAPTCHA, authentication, access controls, rate limits, or similar safeguards.

6.9 Claude Desktop Automation

FR-060 — Desktop Executor

Claude Desktop MUST use a separate DesktopExecutor abstraction.

FR-061 — Application Detection

The executor MUST detect whether the application is installed and available.

FR-062 — Window Activation

The executor MUST identify or activate the relevant application window.

FR-063 — Conversation Selection

The executor MUST support target-specific conversation selection.

FR-064 — Input and Send

The executor MUST support message-entry and send operations where observable controls permit.

FR-065 — Desktop Failure Handling

The executor MUST fail safely on hidden/minimized/missing windows, changed controls, unexpected dialogs, incompatible scaling, or application-update changes.

6.10 Execution Result Management

FR-066 — Success

A run is successful only when the system has sufficient evidence that the intended action was submitted and target-side completion was observed according to the adapter's completion contract.

FR-067 — Partial Success

The system MAY use `PARTIALLY_SUCCEEDED` when a defined subset of expected operations completed but full completion could not be confirmed.

FR-068 — Retryable Failure

The system MUST distinguish retryable failures from permanent failures.

FR-069 — Manual Action

The system MUST provide `MANUAL_ACTION_REQUIRED` or equivalent when safe automatic continuation is impossible.

FR-070 — Execution Evidence

Each attempt MUST preserve structured evidence sufficient to diagnose what happened without storing unnecessary sensitive content.

FR-071 — Full Response Storage

Full AI response storage is not required in the initial release.

6.11 Dashboard and Monitoring

FR-072 — Dashboard

The dashboard MUST display at least:

- total tasks;
- scheduled tasks;
- queued tasks;
- running task;
- recent successes;
- recent failures;
- tasks requiring attention;

- next scheduled task;
- target status;
- scheduler status.

FR-073 — Task History

The UI MUST support execution history review.

FR-074 — Task Detail

The UI MUST display task definition, schedule, current state, and recent runs.

FR-075 — Retry Controls

Eligible failed runs MUST provide a manual retry action.

FR-076 — Target Health

Target readiness and last validation state MUST be visible.

6.12 Settings and Configuration

FR-077 — Target Paths

The UI or configuration system MUST support target executable and profile path configuration.

FR-078 — Timeouts

Timeouts MUST be configurable.

FR-079 — Retry Policy

Retry parameters MUST be configurable.

FR-080 — File Restrictions

File extension, size, and count restrictions MUST be configurable.

FR-081 — Logging

Log level and retention MUST be configurable.

FR-082 — Sleep Policy

Automatic sleep policy MUST be configurable.

6.13 Startup, Shutdown, and Packaging

FR-083 — First Launch

First launch MUST initialize required directories and database state.

FR-084 — Startup Recovery

Startup MUST detect stale execution states and recover them according to recovery rules.

FR-085 — Graceful Shutdown

The system MUST prevent unsafe interruption of a running target operation where possible.

FR-086 — Windows Restart Recovery

After Windows restart, the application MUST restore persisted state and scheduling.

FR-087 — Backup and Restore

The application MUST provide local database backup and restore procedures.

FR-088 — Diagnostics

The application MUST support collection of a safe diagnostic bundle excluding credentials and sensitive content.

7. Non-Functional Requirements

7.1 Reliability

NFR-001

The application MUST persist task state before executing externally observable operations.

NFR-002

A process crash MUST NOT silently delete task definitions.

NFR-003

A recovered RUNNING task MUST be classified according to the last known execution evidence.

7.2 Availability

NFR-004

The local UI SHOULD remain available while the scheduler/agent is operating.

NFR-005

Scheduler operation MUST NOT depend on the Web UI browser tab remaining open.

7.3 Performance

NFR-006

Local API requests for ordinary CRUD operations SHOULD complete within 500 ms under normal single-user conditions.

NFR-007

The scheduler SHOULD detect due work within 15 seconds when the computer and agent are awake.

NFR-008

Task start should normally occur within ± 60 seconds after the scheduled time when Windows is awake and the agent is healthy.

Wake-time accuracy remains dependent on Windows and hardware behavior.

7.4 Maintainability

NFR-009

Target-specific automation MUST be isolated behind adapters.

NFR-010

Database access MUST be isolated behind a persistence layer.

NFR-011

Database schema changes MUST use migration tooling.

7.5 Extensibility

NFR-012

New targets SHOULD be addable without rewriting scheduler logic.

NFR-013

New task types SHOULD be implementable without breaking existing task-type isolation.

7.6 Usability

NFR-014

The main task creation flow SHOULD be understandable without technical knowledge.

NFR-015

Validation errors MUST identify the field and corrective action.

7.7 Observability

NFR-016

Every execution attempt MUST have a unique run/attempt identifier.

NFR-017

Major state transitions MUST be logged.

7.8 Privacy

NFR-018

The system MUST keep file contents on the local filesystem until intentionally uploaded to the selected target.

NFR-019

Session tokens, cookies, passwords, and sensitive file content MUST NOT be written to ordinary logs.

7.9 Security

NFR-020

The local API MUST bind to localhost by default.

NFR-021

No credentials may be hard-coded in source code.

NFR-022

State-changing local API operations SHOULD require same-origin validation or a local-request protection mechanism.

7.10 Compatibility

NFR-023

The system MUST support the project's validated Windows version and dependency set.

NFR-024

Target-specific behavior **MUST** be validated after target application updates.

7.11 Recoverability

NFR-025

Database backup **MUST** be possible without stopping the whole computer.

NFR-026

The system **SHOULD** recover from normal application restart without losing queued tasks.

7.12 Data Integrity

NFR-027

Foreign-key constraints **MUST** be enforced.

NFR-028

Task claiming **MUST** be transactional.

NFR-029

Timestamps **MUST** use UTC for persistence and local time for display.

NFR-030

The system **SHOULD** use SQLite WAL mode under normal operation.

7.13 Recommended Capacity Defaults

Because the planning baseline does not define exact limits, the following are Recommended Defaults and **MUST** remain configurable:

Setting	Recommended Default
Maximum files per task	10
Maximum individual file size	50 MB
Maximum aggregate attachment size	250 MB
Default maximum attempts	3
Base retry delay	60 seconds
Maximum retry delay	30 minutes

Log retention	30 days
Execution history retention	180 days
Daily database backups retained	7
Scheduler polling interval	15 seconds
Sleep inactivity grace period	2 minutes
Default execution worker count	1
Recommended maximum active task definitions	50,000
Recommended maximum execution records	100,000

These limits are application-level safeguards and do not represent external AI-service upload or usage limits.

8. System Architecture

8.1 Component Overview

Local Web UI

Responsibilities:

- task creation;
- task editing;
- dashboard;
- target configuration;
- scheduling configuration;
- status display;
- history and logs;
- manual retry/resume/cancel controls.

Prohibited responsibility:

- direct manipulation of browser sessions;
- direct database writes bypassing the backend.

FastAPI Backend/API

Responsibilities:

- validate requests;
- manage tasks;
- manage targets;
- expose health information;
- orchestrate database operations.

Prohibited responsibility:

- executing GUI automation inside request handlers;
- performing long-running automation synchronously inside HTTP requests.

Task Service

Responsibilities:

- task business rules;
- task state validation;
- scheduling relationships;
- task-version handling.

Scheduler Service

Responsibilities:

- identify due tasks;
- calculate next runs;
- create queue entries;
- coordinate Windows Task Scheduler wake configuration.

Queue Engine

Responsibilities:

- claim due work;
- enforce ordering;

- prevent duplicates;
- coordinate single-worker execution.

Automation Agent

Responsibilities:

- execute target interactions;
- manage browser/desktop adapters;
- validate local files;
- collect evidence;
- return structured results.

Target Adapter Interface

Responsibilities:

- abstract differences between browser and desktop targets.

Browser Executor

Responsibilities:

- Chrome/Brave/Edge control through Playwright.

Desktop Executor

Responsibilities:

- Claude Desktop control through Windows UI Automation and/or pywinauto.

File Material Service

Responsibilities:

- path normalization;
- path validation;
- metadata;
- attachment preparation.

SQLite Repository

Responsibilities:

- durable local storage;
- transactions;
- migrations;
- indexed queries.

Logging and Monitoring Service

Responsibilities:

- structured application logs;
- execution events;
- audit information;
- safe diagnostic data.

Configuration Service

Responsibilities:

- defaults;
- validated configuration;
- environment-specific values;
- runtime settings.

Windows Task Scheduler Integration

Responsibilities:

- configuring wake-up tasks;
- starting local agent;
- coordinating due-task execution.

Windows Power Management Integration

Responsibilities:

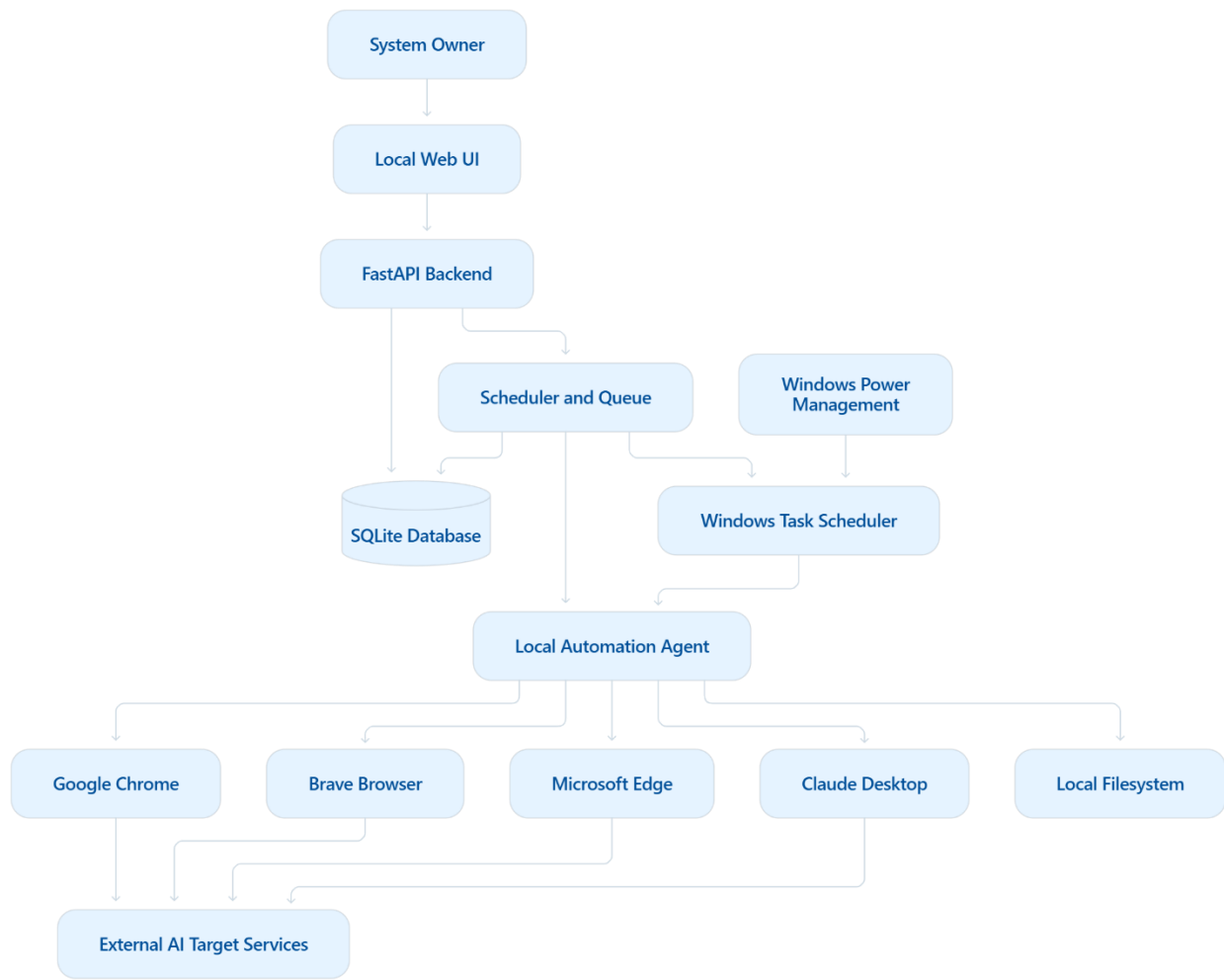
- requesting sleep only when safe;
- preserving active execution;
- recording power-operation failures.

Local Filesystem

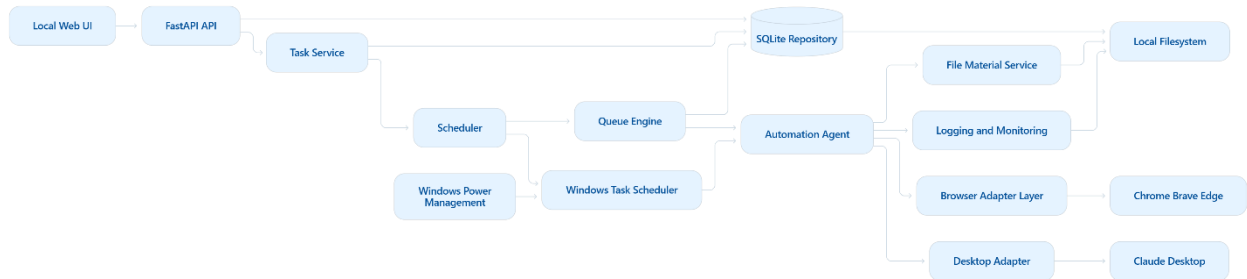
Responsibilities:

- source files;
- logs;
- database;
- backups;
- screenshots;
- configuration.

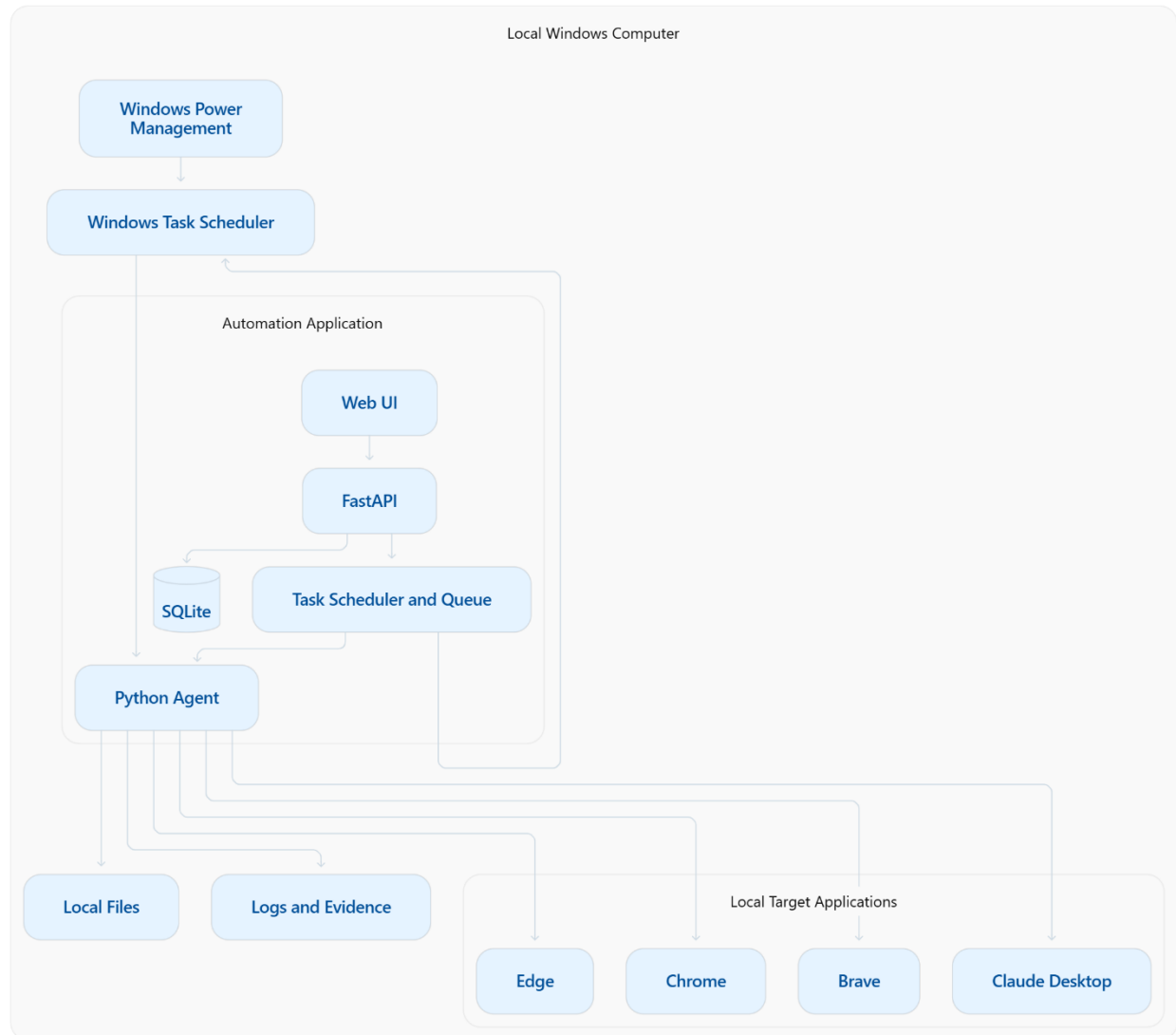
8.2 System Context Diagram



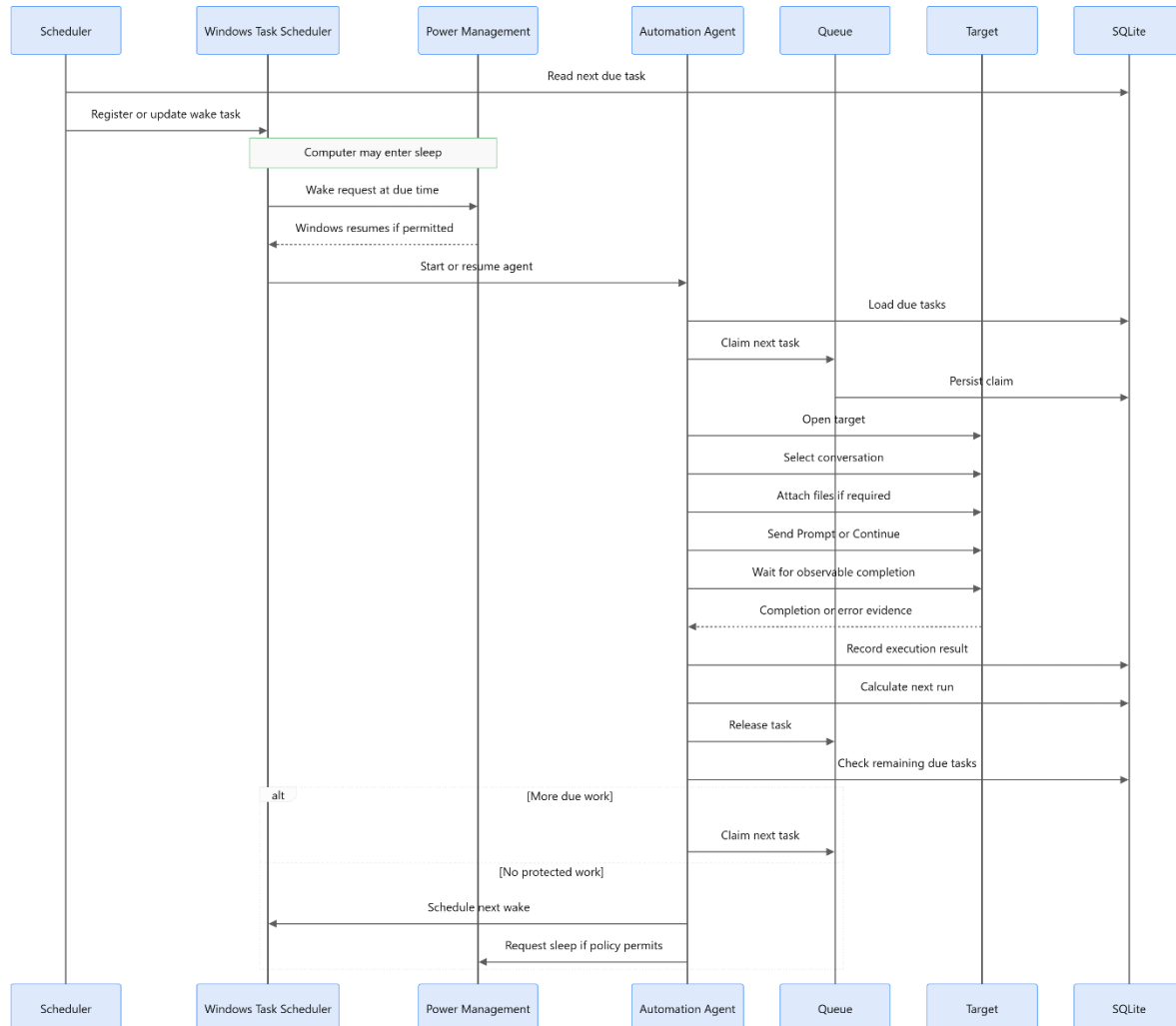
8.3 Container/Component Architecture Diagram



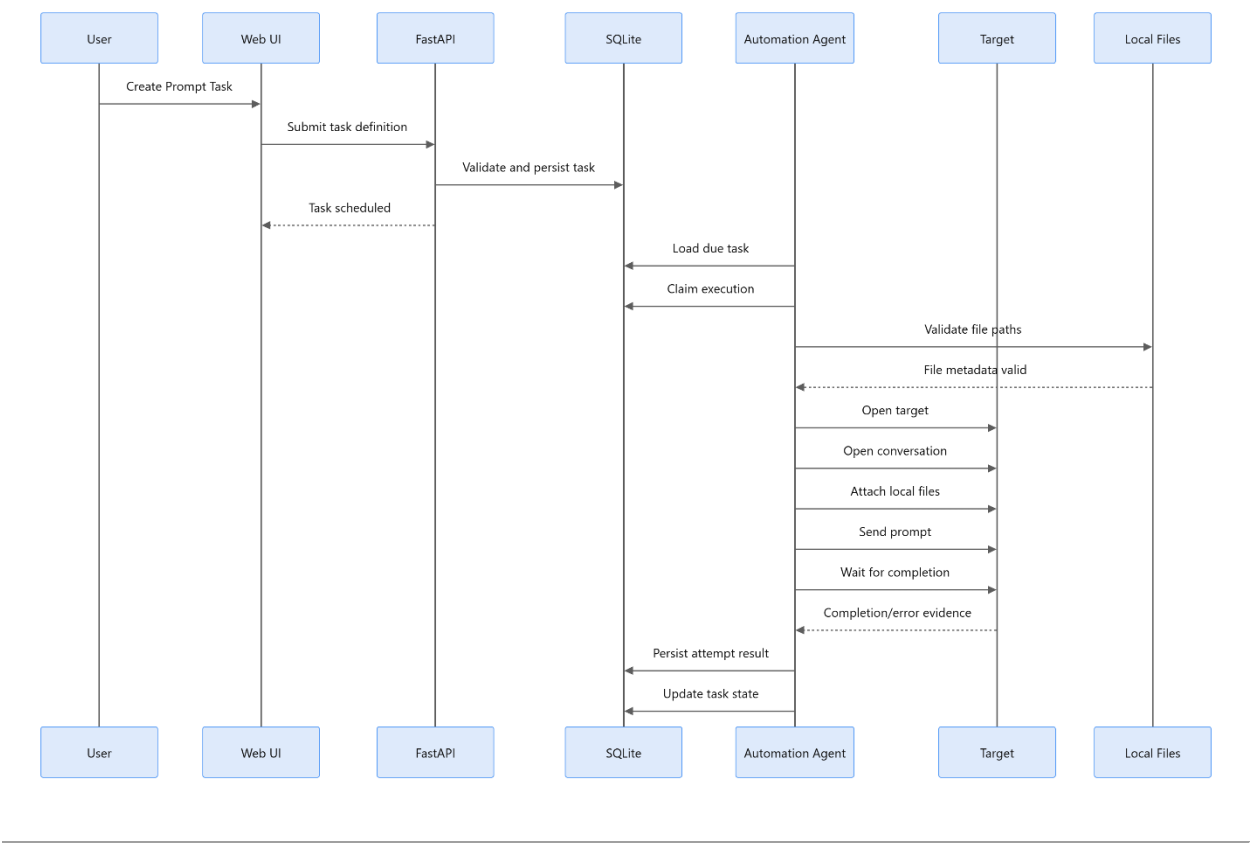
8.4 Deployment Diagram



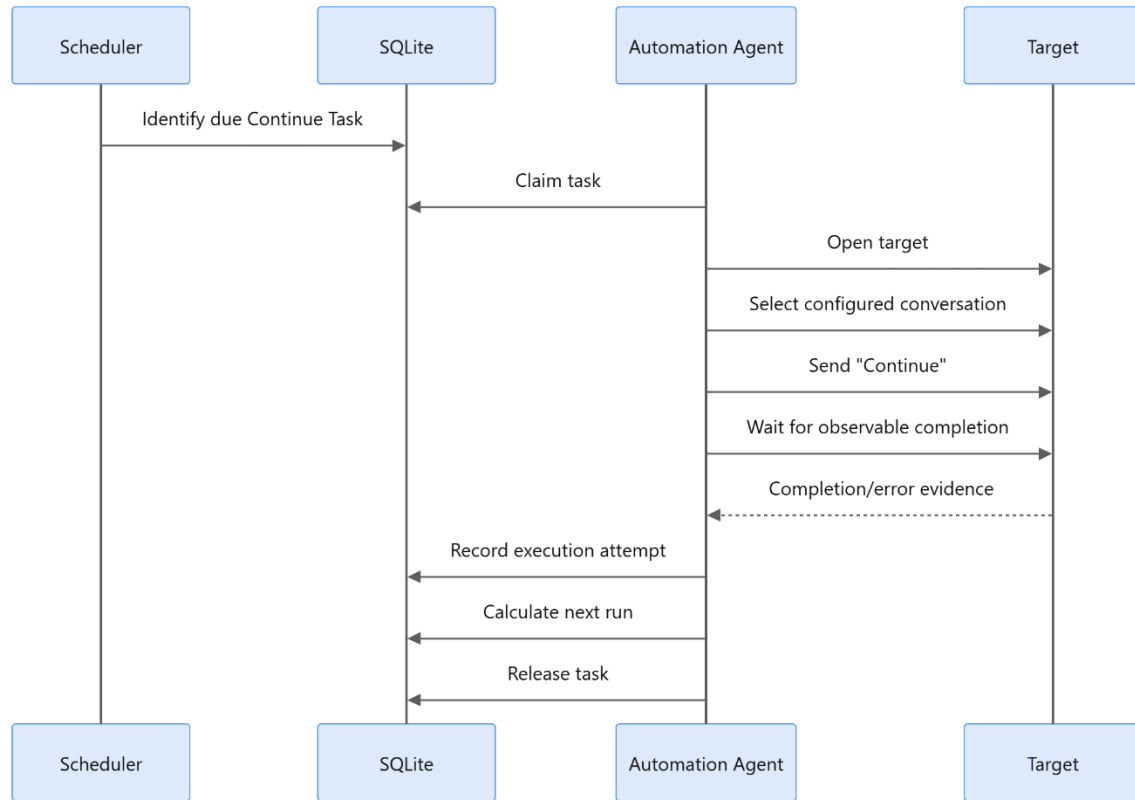
8.5 Scheduling and Wake/Sleep Sequence Diagram



8.6 Scheduling and Wake/Sleep Sequence Diagram



8.7 Continue Task Execution Sequence Diagram



8.8 Architectural Principles

1. **Local-first** — all required components run locally.
2. **Modularity** — UI, API, scheduler, queue, persistence, and automation are separate concerns.
3. **Adapter isolation** — target-specific UI behavior stays inside adapters.
4. **Persistent queue state** — scheduled work survives restart.
5. **Idempotency where possible** — repeated internal operations should not create duplicate runs.
6. **Secure session handling** — use legitimate existing sessions; never store passwords.
7. **No hard-coded credentials** — secrets remain outside source.
8. **Recoverable execution** — failures become observable states.
9. **Testability** — components provide testable contracts.

10. **Explicit state transitions** — task definition lifecycle and execution-run lifecycle remain distinct.

11. **Cautious power management** — sleep occurs only when safe.

12. **No mandatory cloud dependency** — the architecture remains operational locally.

9. Technology Specification

Layer	Recommended Technology	Role	Justification	Alternatives	Decision Status
Language	Python	Core implementation	Strong Windows automation ecosystem	C#	Confirmed
API	FastAPI	Local REST API	Lightweight, typed, easy to test	Flask	Confirmed
Database	SQLite	Local persistence	Zero external server dependency	MariaDB/MySQL	Confirmed
ORM	SQLAlchemy	Persistence abstraction	Clean model/repository layer	SQLModel	Recommended Default
Migrations	Alembic	Schema migration	Mature SQLAlchemy integration	Custom migrations	Recommended Default
Browser Automation	Playwright	Browser control	Persistent contexts and strong browser automation model	Selenium	Confirmed

Desktop Automation	Windows Automation / pywinauto	Claude Desktop	Native Windows control	WinAppDriver or lower-level UIA libraries	Confirmed Direction
Scheduling	Internal scheduler	Task timing	Durable local business scheduling	APScheduler-compatible implementation	Recommended Default
OS Scheduling	Windows Task Scheduler	Wake/start	Native Windows scheduling and wake support	Custom Windows service	Confirmed
Frontend	HTML/CSS/JavaScript	Initial UI	Minimal local deployment complexity	React	Confirmed
Testing	pytest	Python testing	Mature ecosystem	unittest	Recommended Default
API Testing	FastAPI TestClient/httpx	API verification	Local deterministic testing	Requests-based tests	Recommended Default
Browser Testing	Playwright test patterns	Real browser tests	Same automation technology	Selenium	Recommended Default
Logging	Python logging + structured formatter	Logs	Standard library plus structured output	loguru	Recommended Default
Configuration	TOML/YAML + environment overrides	Configuration	Human-readable	JSON	Recommended Default

Packaging	PyInstaller	Windows packaging	Practical standalone packaging	Nuitka	Recommended Default
Version Control	Git	Change management	Required for iterative development	None	Recommended Default

Heavy infrastructure such as Redis, Docker, PostgreSQL, or a cloud queue is intentionally excluded from the initial architecture because the system is a single-user local Windows application. Such infrastructure remains future-scope only.

10. Browser Target Model

10.1 Generic Target Contract

Each target **MUST** expose a logical contract containing:

```
target_id
display_name
target_type
enabled
executable_path
profile_path
capabilities
health_state
last_validated_at
default_timeout
configuration
```

A target adapter **SHOULD** expose:

```
health_check()
open_target()
open_conversation()
attach_files()
send_prompt()
send_continue()
wait_for_response()
detect_completion()
detect_error()
collect_execution_evidence()
release_target()
```

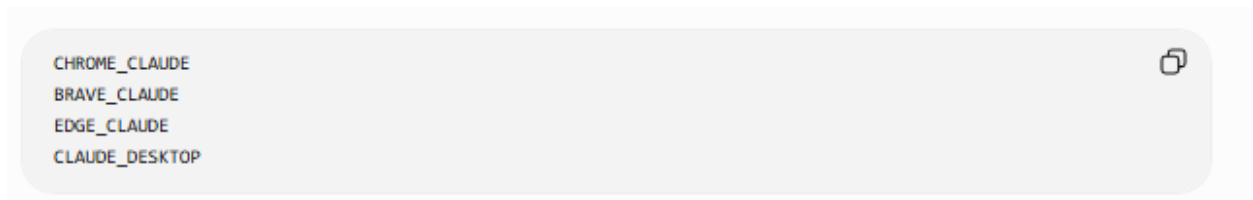
10.2 Capability Matrix

Capability	Chrome	Brave	Edge	Claude Desktop
Automation mechanism	Playwright	Playwright with configured executable/profile	Playwright	Windows UI Automation / pywinauto
Executable configurable	Yes	Yes	Yes	Yes
Persistent profile	Yes	Yes	Yes	Native application state
Prompt support	Required	Required	Required	Required
Continue support	Required	Required	Required	Required
File attachment	Adapter-dependent	Adapter-dependent	Adapter-dependent	Adapter-dependent
Conversation navigation	Browser locator strategy	Browser locator strategy	Browser locator strategy	Desktop UI strategy
Wake/sleep compatibility	Requires usable interactive session	Requires usable interactive session	Requires usable interactive session	Requires usable interactive session
Expected reliability	Target UI dependent	Target UI dependent	Target UI dependent	More UI-sensitive
Special limitation	Browser changes UI	Executable/profile variability	Browser policy/UI changes	Native UI and window-state sensitivity

Manual setup	Automation profile + authentication	Automation profile + authentication	Automation profile + authentication	Existing application session
--------------	-------------------------------------	-------------------------------------	-------------------------------------	------------------------------

10.3 Target IDs

Recommended initial identifiers:



These IDs are internal and MUST NOT assume that a vendor account identifier is known.

10.4 Target Configuration

Each target SHOULD store:

- target ID;
- display name;
- target type;
- executable path;
- profile path;
- enabled state;
- health state;
- capability flags;
- last validation time;
- default timeout;
- configuration JSON;
- optional account label defined by the user.

Account Label

The user interface MAY show a user-defined account label such as:

Claude Research Account

The system **MUST NOT** infer account identity from a login credential.

10.5 Executable Path Discovery

Executable paths **MUST** be configurable.

The system **SHOULD** provide:

1. automatic discovery where practical;
 2. browse/select executable;
 3. manual path override;
 4. validation that the configured executable exists.
-

11. Task Data Model

The system **MUST** distinguish:

- Task Definition;
- Schedule;
- Queue Entry;
- Execution Run;
- Execution Attempt;
- File Reference;
- Target;
- Conversation;
- Execution Event;
- System Setting.

11.1 Task Definition

Core fields:

Field	Meaning
task_id	Stable unique task identifier
task_name	Human-readable task name
task_type	PROMPT or CONTINUE
target_id	Target association
conversation_id	Local conversation configuration reference
prompt_text	Prompt Task instruction
continuation_text	Default Continue value
priority	Queue priority
status	Task lifecycle state
enabled	Whether scheduler may execute it
schedule_id	Schedule association
max_attempts	Retry limit
retry_policy_id	Retry policy
created_at	Task creation timestamp
updated_at	Last task-definition update
archived_at	Optional archive time
version	Optimistic locking/version field

11.2 Schedule

Field	Meaning
schedule_id	Unique schedule ID
schedule_type	ONE_TIME, RECURRING, EVERY_N_HOURS
start_at	First execution

interval_hours	Interval for hourly recurrence
time_zone	User display time zone
fixed_rate_or_delay	Recurrence semantics
next_run_at	Next calculated execution
last_run_at	Last actual run
enabled	Schedule state
expires_at	Optional expiry

11.3 Queue Entry

Field	Meaning
queue_id	Unique queue entry ID
task_id	Associated task
execution_run_id	Associated run
due_at	Queue due time
priority	Effective priority
status	QUEUED, CLAIMED, COMPLETED, CANCELLED
claimed_at	Claim timestamp
worker_id	Local worker identifier
created_at	Queue-entry creation time

11.4 Execution Run

One Task Definition may have many Execution Runs.

Field	Meaning
run_id	Unique execution ID
task_id	Source task definition
scheduled_for	Intended run time
started_at	Start time

finished_at	End time
status	Run-level status
attempt_count	Attempts performed
target_snapshot	Target configuration snapshot
task_version	Task definition version used
result_summary	Safe summary
uncertain_outcome	Boolean
created_at	Run record creation

11.5 Execution Attempt

Field	Meaning
attempt_id	Unique attempt ID
run_id	Parent execution run
attempt_number	Sequence
started_at	Attempt start
finished_at	Attempt end
status	SUCCESS / FAILED / etc.
error_code	Stable error code
evidence_path	Optional safe evidence
retryable	Boolean
completion_confidence	HIGH / MEDIUM / UNKNOWN
created_at	Record creation

11.6 File Reference

Field	Meaning
task_file_id	Unique file reference
task_id	Parent task

file_path	Normalized absolute path
file_name	Display name
extension	Extension
size_bytes	Current known size
modified_at	Current known modification timestamp
sha256	Optional content fingerprint
display_order	Attachment order
enabled	Whether to attach

11.7 Target

Target fields are defined in Section 10.

11.8 Conversation

The local conversation record SHOULD include:

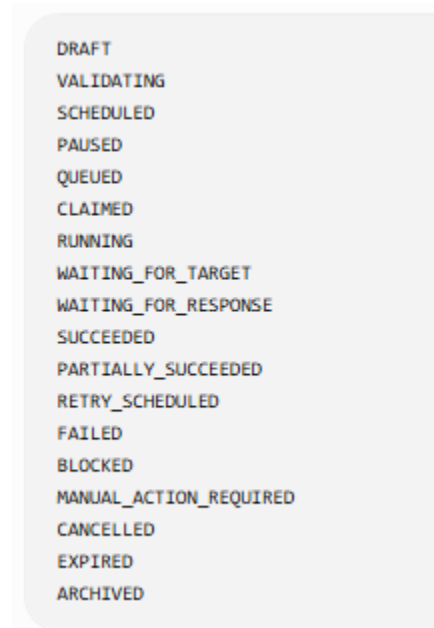
Field	Meaning
conversation_id	Internal stable ID
target_id	Associated target
display_name	Friendly name
locator_type	URL, route, title-search, etc.
locator_value	Stored locator
locator_metadata	Target-specific JSON
enabled	Whether usable
last_validated_at	Last health/identity validation
health_state	READY, UNKNOWN, INVALID

No assumption is made that the external AI provider exposes a permanent API conversation ID.

12. Task Lifecycle and State Machine

12.1 Final Task States

The initial release uses the following task-definition states:



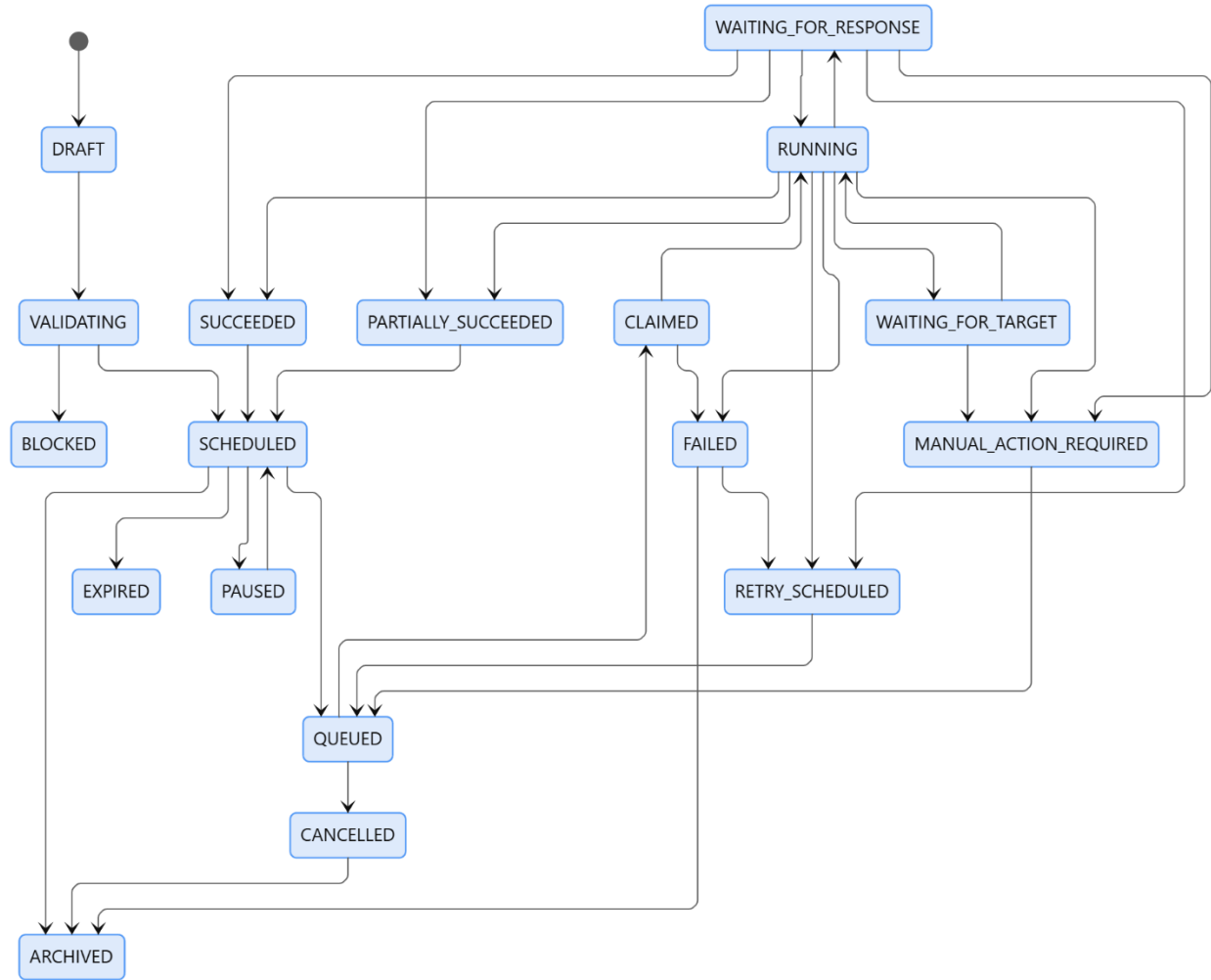
Not every state is terminal.

12.2 State Meanings

State	Meaning	Terminal
DRAFT	Not yet submitted/scheduled	No
VALIDATING	Being validated	No
SCHEDULED	Waiting for due time	No
PAUSED	Temporarily disabled	No
QUEUED	Ready to execute	No
CLAIMED	Reserved by worker	No
RUNNING	Target interaction in progress	No
WAITING_FOR_TARGET	Waiting for target readiness	No

WAITING_FOR_RESPONSE	Waiting for observable completion	No
SUCCEEDED	Latest execution completed successfully	Depending on recurrence
PARTIALLY_SUCCEEDED	Defined partial outcome	No for recurring task
RETRY_SCHEDULED	Retry is planned	No
FAILED	Run failed permanently	Depends on task schedule
BLOCKED	Prerequisite prevents execution	No
MANUAL_ACTION_REQUIRED	User intervention required	No
CANCELLED	Execution/task cancelled	Yes for current schedule
EXPIRED	One-time task expired without execution	Yes
ARCHIVED	Task retained for history only	Yes

12.3 State Diagram



12.4 Invalid Transitions

The following are invalid unless a special recovery operation explicitly creates a new run:

- RUNNING → DRAFT
- SUCCEEDED → RUNNING directly
- CANCELLED → RUNNING
- ARCHIVED → RUNNING
- EXPIRED → RUNNING
- ARCHIVED → SCHEDULED

12.5 User Actions by State

State	Edit	Pause	Resume	Cancel	Manual Run
DRAFT	Yes	No	No	Yes	Yes after validation
SCHEDULED	Yes	Yes	No	Yes	Yes
PAUSED	Limited	No	Yes	Yes	Yes with explicit action
QUEUED	Limited	Yes	No	Yes	No duplicate
CLAIMED	No	No	No	Only safe cancellation	No
RUNNING	No	No	No	Safe stop only if supported	No
WAITING_FOR_TARGET	No	No	No	Yes	No
WAITING_FOR_RESPONSE	No	No	No	Yes, with uncertain-outcome warning	No
RETRY_SCHEDULED	Limited	Yes	No	Yes	Yes
FAILED	Yes	No	Yes by retry/requeue	Yes	Yes
MANUAL_ACTION_REQUIRED	Yes	No	Yes after resolution	Yes	Yes

CANCELLED	Read-only	No	No	No	Only clone/manual new run
ARCHIVED	No	No	No	No	No

12.6 Crash Recovery

At startup:

1. Identify nonterminal CLAIMED and RUNNING tasks.
2. Inspect latest execution evidence.
3. Determine whether completion is known.
4. If send status is uncertain, transition to MANUAL_ACTION_REQUIRED.
5. If no external action was initiated, return to QUEUED or RETRY_SCHEDULED.
6. Never blindly resend a potentially submitted prompt.

12.7 Idempotency

Internal queue and database operations SHOULD be idempotent.

External send actions cannot be assumed idempotent unless the target itself exposes a reliable deduplication mechanism.

Where outcome is unknown:

completion_confidence = UNKNOWN

and automatic duplicate send is prohibited by default.

12.8 Task Definition vs Execution Run

Editing a task definition MUST NOT retroactively rewrite historical run records.

Every Execution Run MUST reference the task-definition version that was executed.

13. Database Requirements

13.1 Tables

Required logical tables:

1. tasks
2. schedules
3. targets
4. conversations
5. task_files
6. queue_entries
7. execution_runs
8. execution_attempts
9. execution_events
10. retry_policies
11. system_settings
12. schema_migrations

13.2 tasks

Column	SQLite Type	Null	Key/Constraint
task_id	TEXT	NO	PK
task_name	TEXT	NO	
task_type	TEXT	NO	CHECK
target_id	TEXT	NO	FK
conversation_id	TEXT	NO	FK
prompt_text	TEXT	YES	Required for Prompt Task
continuation_text	TEXT	YES	Defaults to Continue

priority	INTEGER	NO	Default 100
status	TEXT	NO	CHECK
enabled	INTEGER	NO	0/1
schedule_id	TEXT	NO	FK
retry_policy_id	TEXT	YES	FK
max_attempts	INTEGER	NO	Positive
current_attempt	INTEGER	NO	Default 0
version	INTEGER	NO	Optimistic lock
created_at	TEXT	NO	UTC
updated_at	TEXT	NO	UTC
archived_at	TEXT	YES	UTC

13.3 schedules

Column	SQLite Type	Null	Key/Constraint
schedule_id	TEXT	NO	PK
schedule_type	TEXT	NO	CHECK
start_at	TEXT	NO	UTC
interval_hours	INTEGER	YES	Positive when used
time_zone	TEXT	NO	
recurrence_mode	TEXT	NO	FIXED_RATE/FIXED_DELAY
next_run_at	TEXT	YES	UTC
last_run_at	TEXT	YES	UTC
enabled	INTEGER	NO	
expires_at	TEXT	YES	UTC
created_at	TEXT	NO	UTC
updated_at	TEXT	NO	UTC

13.4 targets

Column	SQLite Type	Null	Key/Constraint
target_id	TEXT	NO	PK
display_name	TEXT	NO	UNIQUE recommended
target_type	TEXT	NO	BROWSER/DESKTOP
browser_type	TEXT	YES	
executable_path	TEXT	YES	
profile_path	TEXT	YES	
capabilities_json	TEXT	NO	JSON
enabled	INTEGER	NO	
health_state	TEXT	NO	
last_validated_at	TEXT	YES	UTC
configuration_json	TEXT	YES	JSON
created_at	TEXT	NO	UTC
updated_at	TEXT	NO	UTC

13.5 conversations

Column	SQLite Type	Null	Key/Constraint
conversation_id	TEXT	NO	PK
target_id	TEXT	NO	FK
display_name	TEXT	NO	
locator_type	TEXT	NO	
locator_value	TEXT	NO	
locator_metadata_json	TEXT	YES	JSON
enabled	INTEGER	NO	
health_state	TEXT	NO	

last_validated_at	TEXT	YES	UTC
created_at	TEXT	NO	UTC
updated_at	TEXT	NO	UTC

13.6 task_files

Column	SQLite Type	Null	Key/Constraint
task_file_id	TEXT	NO	PK
task_id	TEXT	NO	FK
file_path	TEXT	NO	
file_name	TEXT	NO	
extension	TEXT	NO	
size_bytes	INTEGER	YES	
modified_at	TEXT	YES	UTC
sha256	TEXT	YES	
display_order	INTEGER	NO	
enabled	INTEGER	NO	

13.7 queue_entries

Column	SQLite Type	Null	Key/Constraint
queue_id	TEXT	NO	PK
task_id	TEXT	NO	FK
execution_run_id	TEXT	NO	FK
due_at	TEXT	NO	UTC
priority	INTEGER	NO	
status	TEXT	NO	
claimed_at	TEXT	YES	UTC
worker_id	TEXT	YES	

created_at	TEXT	NO	UTC
updated_at	TEXT	NO	UTC

13.8 execution_runs

Column	SQLite Type	Null	Key/Constraint
run_id	TEXT	NO	PK
task_id	TEXT	NO	FK
scheduled_for	TEXT	NO	UTC
started_at	TEXT	YES	UTC
finished_at	TEXT	YES	UTC
status	TEXT	NO	
attempt_count	INTEGER	NO	
task_version	INTEGER	NO	
target_snapshot_json	TEXT	YES	JSON
result_summary	TEXT	YES	
uncertain_outcome	INTEGER	NO	0/1
created_at	TEXT	NO	UTC

13.9 execution_attempts

Column	SQLite Type	Null	Key/Constraint
attempt_id	TEXT	NO	PK
run_id	TEXT	NO	FK
attempt_number	INTEGER	NO	
started_at	TEXT	NO	UTC
finished_at	TEXT	YES	UTC
status	TEXT	NO	

error_code	TEXT	YES	
evidence_path	TEXT	YES	
retryable	INTEGER	NO	
completion_confidence	TEXT	NO	HIGH/MEDIUM/UNKNOWN
created_at	TEXT	NO	UTC

13.10 execution_events

Column	SQLite Type	Null	Key/Constraint
event_id	TEXT	NO	PK
run_id	TEXT	YES	FK
attempt_id	TEXT	YES	FK
event_type	TEXT	NO	
timestamp	TEXT	NO	UTC
component	TEXT	NO	
severity	TEXT	NO	
safe_context_json	TEXT	YES	JSON

13.11 retry_policies

Column	SQLite Type	Null	Key/Constraint
retry_policy_id	TEXT	NO	PK
max_attempts	INTEGER	NO	
base_delay_seconds	INTEGER	NO	
max_delay_seconds	INTEGER	NO	
exponential	INTEGER	NO	
jitter	INTEGER	NO	
retryable_codes_json	TEXT	YES	JSON
created_at	TEXT	NO	UTC

updated_at	TEXT	NO	UTC
------------	------	----	-----

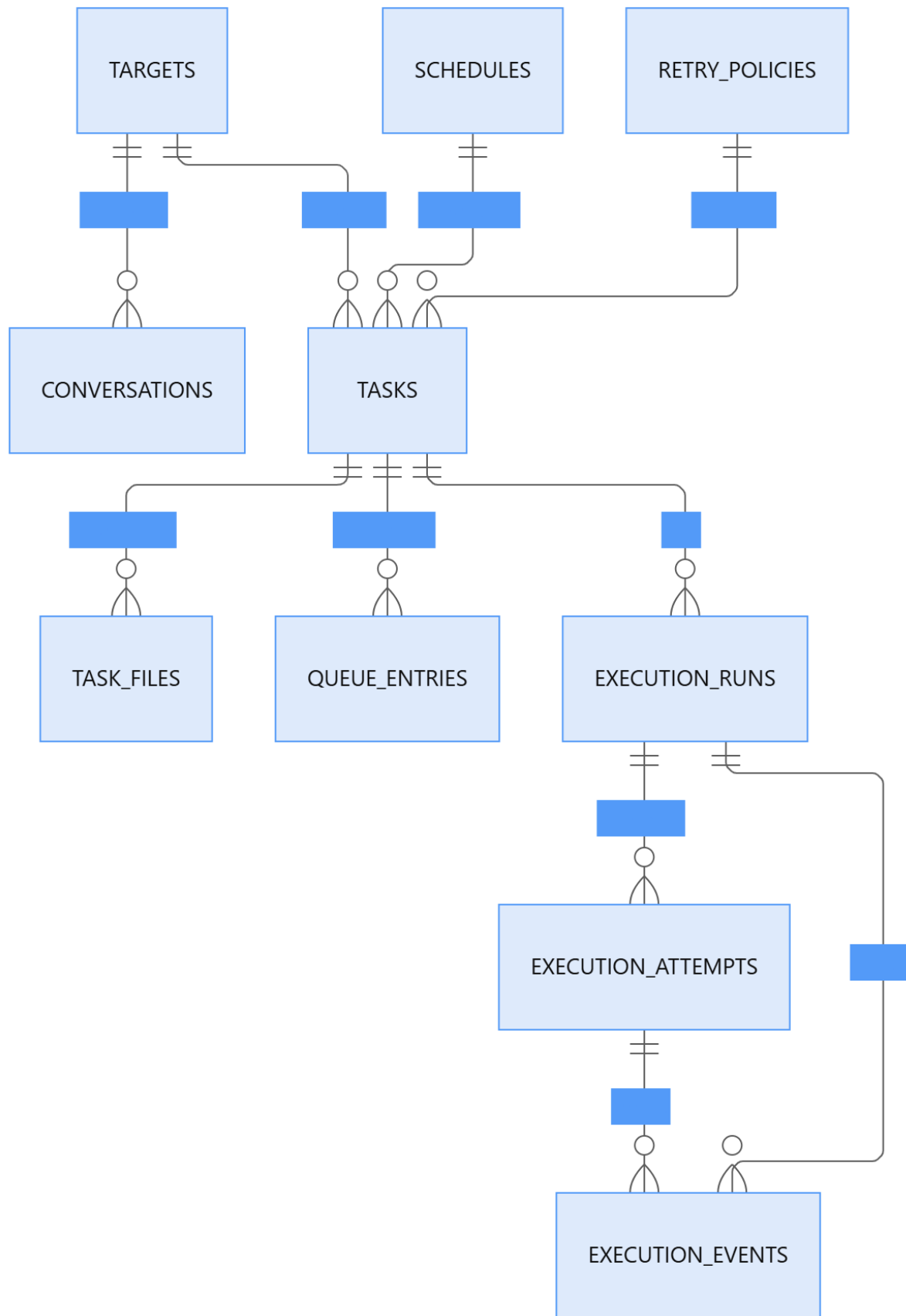
13.12 system_settings

Column	SQLite Type	Null	Key/Constraint
setting_key	TEXT	NO	PK
setting_value	TEXT	NO	
value_type	TEXT	NO	
updated_at	TEXT	NO	UTC

13.13 schema_migrations

Managed by Alembic or equivalent.

13.14 Entity Relationship Diagram



13.15 Data Dictionary Rules

- IDs SHOULD use UUID or ULID-like text values.
- timestamps MUST use UTC.
- boolean values MUST use SQLite integer 0/1.
- enumerated values MUST be stored as TEXT with validation.
- JSON fields MUST contain schema-valid JSON.
- path strings MUST be normalized before persistence.

13.16 Indexing Strategy

Recommended indexes:

- tasks(status, enabled, updated_at)
- tasks(target_id, status)
- schedules(next_run_at, enabled)
- queue_entries(status, due_at, priority)
- execution_runs(task_id, started_at)
- execution_attempts(run_id, attempt_number)
- execution_events(run_id, timestamp)
- task_files(task_id, display_order)

13.17 Transaction and Concurrency Rules

SQLite MUST use:

- foreign-key enforcement;
- WAL mode;
- explicit transactions for task claiming;
- a short busy timeout;
- deterministic state transitions.

The queue worker MUST claim work transactionally so another process cannot claim the same queue entry.

13.18 Migration Strategy

All schema changes **MUST** be versioned through Alembic or equivalent migration tooling.

A backup **MUST** be taken before production migration.

13.19 Backup and Restore

Recommended default:

- daily backup;
- retain seven daily copies;
- allow user-selected backup directory;
- never store credentials in backups;
- restore **MUST** be a deliberate administrative operation.

13.20 Database Integrity

The system **MUST** reject:

- orphan task records;
- task references to nonexistent targets;
- task references to nonexistent schedules;
- duplicate queue ownership;
- invalid recurrence intervals;
- invalid enum values.

14. API Requirements

The API is local-only by default and **SHOULD** bind to 127.0.0.1.

14.1 Health

Method	Path	Purpose
GET	/api/v1/health	Basic service health
GET	/api/v1/health/dependencies	Database, scheduler, targets

14.2 Tasks

Method	Path	Purpose
GET	/api/v1/tasks	List tasks
POST	/api/v1/tasks	Create task
GET	/api/v1/tasks/{task_id}	Get task
PATCH	/api/v1/tasks/{task_id}	Edit task
POST	/api/v1/tasks/{task_id}/duplicate	Duplicate
POST	/api/v1/tasks/{task_id}/enable	Enable
POST	/api/v1/tasks/{task_id}/disable	Disable
POST	/api/v1/tasks/{task_id}/pause	Pause
POST	/api/v1/tasks/{task_id}/resume	Resume
POST	/api/v1/tasks/{task_id}/run	Manual run
POST	/api/v1/tasks/{task_id}/cancel	Cancel
DELETE	/api/v1/tasks/{task_id}	Delete/archive

14.3 Schedules

Method	Path	Purpose
GET	/api/v1/schedules	List schedules
GET	/api/v1/schedules/{schedule_id}	Get schedule
PATCH	/api/v1/schedules/{schedule_id}	Update schedule

14.4 Targets

Method	Path	Purpose
GET	/api/v1/targets	List targets
POST	/api/v1/targets	Create/configure target
GET	/api/v1/targets/{target_id}	Get target
PATCH	/api/v1/targets/{target_id}	Update target

POST	/api/v1/targets/{target_id}/health-check	Validate target
------	--	-----------------

14.5 Conversations

Method	Path	Purpose
GET	/api/v1/conversations	List configured conversations
POST	/api/v1/conversations	Add conversation
GET	/api/v1/conversations/{conversation_id}	Get configuration
PATCH	/api/v1/conversations/{conversation_id}	Edit
POST	/api/v1/conversations/{conversation_id}/validate	Validate

14.6 Files

Method	Path	Purpose
POST	/api/v1/files/validate	Validate path
POST	/api/v1/files/metadata	Read metadata
POST	/api/v1/files/verify	Verify stored reference

The API MUST NOT upload or store binary file content in the database.

14.7 Execution

Method	Path	Purpose
GET	/api/v1/executions	List runs
GET	/api/v1/executions/{run_id}	Run details
GET	/api/v1/executions/{run_id}/attempts	Attempt history
POST	/api/v1/executions/{run_id}/retry	Manual retry

14.8 Logs

Method	Path	Purpose
GET	/api/v1/logs	Filtered logs
GET	/api/v1/logs/events	Execution events

POST	/api/v1/logs/diagnostic-bundle	Create safe diagnostic bundle
------	--------------------------------	-------------------------------

14.9 Scheduler Controls

Method	Path	Purpose
GET	/api/v1/scheduler/status	Scheduler state
POST	/api/v1/scheduler/wakeup-sync	Recalculate Windows wake schedule
POST	/api/v1/scheduler/recalculate	Recalculate next runs

14.10 Agent Controls

Method	Path	Purpose
GET	/api/v1/agent/status	Local agent health
POST	/api/v1/agent/restart-request	Controlled restart request

14.11 API Validation

Requests MUST return consistent errors.

Recommended format:

```
{
  "error_code": "VAL-001",
  "message": "Prompt text is required for Prompt Tasks.",
  "field": "prompt_text",
  "request_id": "..."}

```

14.12 Status Codes

- 200 success;
- 201 created;
- 204 successful no-content;
- 400 invalid request;
- 404 not found;

- 409 state/conflict;
- 422 validation error;
- 500 internal error;
- 503 dependency unavailable.

14.13 API Versioning

All API endpoints MUST use /api/v1/.

14.14 Local Binding

Default binding:

127.0.0.1

LAN exposure is out of scope and requires an explicit security decision.

15. User Interface Requirements

15.1 Dashboard

Purpose

Provide system-wide status.

Display

- agent status;
- scheduler status;
- target health;
- next task;
- current task;
- task counts;
- recent failures;
- tasks requiring attention.

Empty State

Clearly indicate when no tasks exist.

Error State

Show actionable error and diagnostic identifier.

15.2 Task List

Display

- task name;
- task type;
- target;
- status;
- next run;
- enabled state;
- last result.

Actions

- view;
 - edit;
 - duplicate;
 - enable/disable;
 - pause/resume;
 - manual run;
 - cancel;
 - archive.
-

15.3 Create Task

The UI MUST offer two distinct creation modes:

Prompt Task Form

- task name;
- target;
- conversation;
- prompt text;
- file selector;
- file list/order;
- schedule type;
- first run time;
- recurrence interval;
- priority;
- retry policy;
- optional expiration.

Continue Task Form

- task name;
- target;
- conversation;
- fixed continuation text display: Continue;
- start time;
- recurrence mode;
- interval hours;
- priority;

- retry policy;
- optional expiration.

Custom continuation text is omitted from the initial UI.

15.4 Task Details

Must display:

- task definition;
- task version;
- current status;
- schedule;
- files;
- target;
- conversation;
- next run;
- recent attempts.

15.5 Execution History

Must display:

- run ID;
- scheduled time;
- actual start;
- end;
- result;
- attempt count;
- error code;
- evidence availability.

15.6 Run Details

Must display safe diagnostic information.

It MUST NOT display:

- passwords;
- session cookies;
- access tokens;
- complete sensitive file contents;
- hidden credentials.

15.7 Targets

Must display:

- target name;
- type;
- enabled state;
- health;
- executable path;
- profile path;
- capabilities;
- last validation.

15.8 Conversations

Must display:

- target;
- display name;
- locator type;

- validation state;
 - last validation time.
-

15.9 Settings

Must expose configurable values approved for runtime changes.

Values requiring restart **MUST** be marked.

15.10 Logs

Must support:

- filtering by task;
 - run;
 - target;
 - severity;
 - date range;
 - error code.
-

15.11 System Health

Must show:

- database status;
 - scheduler status;
 - agent status;
 - target statuses;
 - filesystem health;
 - Windows scheduler integration status;
 - next wake time.
-

15.12 Accessibility

The UI SHOULD:

- use keyboard-accessible controls;
 - have visible focus state;
 - use semantic labels;
 - avoid conveying status only through color;
 - display validation errors in text.
-

16. File-Handling Rules

16.1 Initial Extensions

Recommended default allowlist:

.pdf
.docx
.txt
.md
.csv
.json
.xlsx
.png
.jpg
.jpeg
.webp

The allowlist MUST be configurable.

Actual target capability remains the final authority for whether a file can be attached successfully.

16.2 Path Rules

The system SHOULD normalize paths before storage.

Stored paths SHOULD be absolute local paths.

16.3 Network Paths

UNC/network paths are out of scope by default and MAY be disabled through configuration.

16.4 Removable Drives

Removable drives are allowed only if the path is currently accessible.

16.5 Duplicate Selection

The system SHOULD detect and prevent identical paths from being attached multiple times to the same task unless explicitly allowed.

16.6 Changed Files

If stored metadata differs from the current file, the system SHOULD show a warning.

The task MAY continue if the file remains accessible unless the user-configured policy requires exact hash matching.

16.7 Moved or Deleted Files

Execution MUST stop and enter MANUAL_ACTION_REQUIRED when a required file cannot be resolved.

16.8 File Locking

A temporary sharing/locking issue MUST be treated as a file-access error.

16.9 Zero-Byte Files

Zero-byte files SHOULD be rejected by default.

16.10 Hidden and Temporary Files

The UI SHOULD warn against selecting temporary files and MAY filter them from default file-picker views.

16.11 Maximums

Recommended defaults are defined under NFR-030.

16.12 Path Privacy

File paths SHOULD be masked in user-facing logs where they are not necessary.

16.13 Database Storage

File binaries MUST NOT be stored in SQLite.

16.14 Cleanup

The system MUST NOT automatically delete the user's source files.

Evidence files generated by the automation system MAY be retained or cleaned according to retention policy.

17. Scheduling Rules

17.1 One-Time Tasks

A one-time task executes once.

After success:

SUCCEDED

No new schedule is generated.

If missed:

- execute when possible by default;
- optionally expire based on expires_at.

17.2 Recurring Tasks

Recurring tasks generate future runs.

Default recurrence:

- calendar/fixed-rate for general recurring schedules;
- fixed-delay for every-N-hours Continue tasks.

17.3 Every-N-Hours Continue Tasks

Default behavior:

1. execute first run at configured start_at;
2. if successful, next run is based on the configured interval from completion;

3. if a run is skipped or missed, do not create a burst of accumulated runs;
4. create only the next eligible run.

This behavior is deliberately designed to avoid multiple immediate Continue submissions following a long offline period.

17.4 Timestamp Storage

All persisted timestamps MUST use UTC.

UI MUST display Windows local time.

17.5 Missed One-Time Runs

Default:

- execute at the first safe opportunity;
- if expires_at is passed, transition to EXPIRED.

17.6 Missed Recurring Runs

Default:

- skip stale intervals;
- schedule the next future occurrence.

17.7 Simultaneous Tasks

Tasks due at the same time enter the queue.

Single-worker ordering uses:

1. manual-run override;
2. retry priority;
3. task priority;
4. due time;
5. task creation order.

17.8 Clock Changes

A significant Windows clock adjustment MUST trigger a scheduler recalculation.

17.9 Schedule Edits

Updating an active schedule MUST recalculate next_run_at.

Historical runs remain unchanged.

17.10 Disabled Tasks

Disabled tasks remain stored but cannot enter the execution queue.

17.11 Retry Precedence

Precedence:

1. active execution continues;
2. manually requested explicit run;
3. due retry;
4. normal scheduled due task;
5. future scheduled work.

17.12 Manual Execution

Manual Run SHOULD NOT shift the normal recurring schedule unless the user selects an explicit schedule-reset option.

17.13 Scheduling Pseudocode

on_scheduler_tick():

 recover_stale_queue_claims()

 recalculate_if_clock_changed()

 candidates = find_due_tasks()

 for task in candidates:

 if task.disabled:

 continue

```
if task.blocked:
```

```
    continue
```

```
if task.has_uncertain_outcome:
```

```
    mark_manual_action_required()
```

```
    continue
```

```
if already_queued_or_running(task):
```

```
    continue
```

```
create_queue_entry(task)
```

```
select_next_queue_entry():
```

```
    prefer manual override
```

```
    then retry due
```

```
    then highest priority
```

```
    then earliest due_at
```

```
    then earliest created_at
```

```
after_success(task):
```

```
    if task is one-time:
```

```
        mark_succeeded()
```

```
    else:
```

```
        calculate_next_run()
```

```
after_failure(task):
```

if retryable and attempts_remaining:

 schedule_retry()

else:

 mark_failed_or_manual_action()

18. Windows Sleep and Wake Requirements

18.1 Windows Task Scheduler

The system **MUST** integrate with Windows Task Scheduler for scheduled wake/start behavior.

Windows Task Scheduler supports wake-to-run behavior for scheduled tasks; the final system **MUST** treat wake behavior as an OS/hardware-dependent capability rather than a guarantee.

18.2 Wake Timer Requirement

Wake timers **MUST** be enabled for supported power configurations.

The application **SHOULD** surface whether wake scheduling appears enabled.

18.3 Conditions That May Prevent Wake-Up

Possible external constraints include:

- wake timers disabled;
- BIOS/UEFI limitations;
- driver limitations;
- unsupported sleep state;
- hardware power restrictions;
- battery policy;
- full shutdown instead of sleep;
- lid/power behavior;
- operating-system policy.

These conditions cannot be fully guaranteed by application code.

18.4 AC and Battery

Recommended default:

- automatic scheduled execution requiring reliable wake behavior SHOULD be performed while AC power is connected;
- the system SHOULD expose a configurable require_ac_power policy.

18.5 Startup/Resume

After wake:

1. local application/agent MUST become available;
2. scheduler MUST detect due tasks;
3. stale task state MUST be recovered;
4. queue MUST resume.

18.6 User Session Requirements

GUI automation requires a usable Windows interactive session.

The system MUST NOT attempt to bypass the Windows login screen.

The initial release MUST NOT assume that browser or desktop GUI automation can operate successfully while the Windows session is locked.

If the session is locked or unavailable:

MANUAL_ACTION_REQUIRED or a defined retry state MUST be used.

18.7 Pre-Sleep Checks

The system MUST NOT request sleep while:

- a task is running;
- a retry is immediately pending;
- a target session remains in an active protected state;
- database migration/backup is running;
- a user-configured keep-awake condition is active;
- recent user activity violates the idle policy.

18.8 Post-Execution Sleep

If:

- no due tasks remain;
- no protected operation remains;
- no immediate retry remains;
- user activity is below configured threshold;
- auto-sleep is enabled;

then the system MAY request Windows sleep.

18.9 Sleep Grace Period

Recommended default: 2 minutes of inactivity before an automatic sleep request.

18.10 User Activity Detection

Recent keyboard/mouse interaction SHOULD suppress automatic sleep for the configured grace interval.

18.11 Failure to Sleep

If sleep fails:

- log POWER-SLEEP-001;
- keep the task system operational;
- do not retry sleep in a tight loop;
- notify the user through the dashboard.

18.12 Failure to Wake

If a scheduled task is not observed by the agent after the expected time:

- preserve task state;
- mark the run as delayed/overdue;
- execute on next available agent startup according to missed-run rules.

18.13 Audit

Power events SHOULD be logged with:

- expected wake;
- actual agent startup;
- task execution start;
- sleep request;
- sleep success/failure.

19. Error-Handling Rules

19.1 Error Taxonomy

Error Category	Example Code	Retryable	Default Action
Validation	VAL-001	No	Reject task
Configuration	CFG-001	No	Manual correction
File	FILE-001	Sometimes	Retry or manual action
Target unavailable	TARGET-001	Yes	Retry
Browser launch	AUTO-001	Yes	Retry
Desktop failure	AUTO-010	Yes	Retry
Session expired	TARGET-010	No	Manual action
Conversation missing	TARGET-020	No	Manual action
UI/locator changed	AUTO-020	No by default	Manual action
Attachment failure	AUTO-030	Yes	Retry
Send failure	AUTO-040	Caution	Verify before retry
Response timeout	AUTO-050	Yes	Retry if outcome known
Network error	SYS-010	Yes	Retry
Scheduler error	SCH-001	Yes	Retry/recover
Wake failure	POWER-001	No immediate retry	Overdue recovery
Sleep failure	POWER-010	No	Log/notify
Database locked	DB-001	Yes	Backoff
Database corruption	DB-010	No automatic recovery	Backup/restore
Permission denied	SYS-020	No	Manual action
Cancellation	SYS-030	No	Stop cleanly
Internal error	SYS-999	Depends	Recover/diagnose

19.2 Error Requirements

Every error MUST define:

- stable code;
 - detection method;
 - severity;
 - retryability;
 - state transition;
 - user message;
 - technical log requirement.
-

20. Retry and Recovery Policy

20.1 Default Retry

Recommended default:

- maximum attempts: 3;
- base delay: 60 seconds;
- exponential backoff enabled;
- maximum delay: 30 minutes;
- small randomized jitter.

20.2 Retryable Categories

Default retryable categories:

- temporary browser startup failure;
- temporary network failure;
- temporary file lock;
- transient attachment failure;
- target not yet ready;
- recoverable timeout.

20.3 Non-Retryable Categories

Default non-retryable:

- missing target;
- missing conversation;
- expired authentication;
- unsupported file;
- invalid configuration;
- clear UI-layout mismatch requiring adapter maintenance;
- uncertain send outcome.

20.4 Uncertain Send Outcome

When a send action may have succeeded but completion cannot be confirmed:

1. mark uncertain_outcome = true;
2. DO NOT automatically resend;
3. capture safe evidence;
4. transition to MANUAL_ACTION_REQUIRED.

This protects against duplicate prompt submission.

20.5 Process Crash

A crashed worker MUST NOT leave work permanently blocked.

Startup recovery MUST inspect stale runs.

20.6 Browser Crash

Browser adapter SHOULD:

1. detect crash;
2. release browser control;
3. revalidate profile;
4. retry if the action was definitely not submitted;
5. otherwise mark uncertainty.

20.7 Database Lock

Database lock errors SHOULD use exponential backoff with a bounded retry count.

20.8 Partial Attachment

If only some files were attached:

- do not assume task success;
 - record attachment state;
 - retry only when safe;
 - otherwise request manual review.
-

21. Logging, Audit, and Monitoring

21.1 Structured Log Fields

Every relevant event SHOULD include:

- timestamp;
- request ID;
- task ID;
- run ID;
- attempt ID;
- target ID;
- component;
- severity;
- event type;
- error code;
- safe context.

21.2 Log Categories

Application Log

General runtime behavior.

Execution Event Log

Task/run/attempt transitions.

Audit Log

User actions such as create, edit, pause, retry, cancel.

Target Health Log

Target readiness and validation.

Scheduler Log

Due-task calculations and queue operations.

Power Event Log

Wake/sleep coordination.

21.3 Log Storage

Recommended:

- logs/
 - application.log
 - scheduler.log
 - executor.log
 - target.log
 - power.log
 - audit.log

Execution events may additionally be stored in SQLite.

21.4 Retention

Recommended default:

- 30 days application logs;
- 180 days execution history.

21.5 Redaction

Ordinary logs MUST NOT contain:

- passwords;
- cookies;
- access tokens;
- authentication headers;
- session secrets;
- full file contents;
- complete highly sensitive prompts unless explicitly enabled.

File paths MAY be masked in diagnostic outputs.

21.6 Diagnostic Bundle

A diagnostic bundle MAY contain:

- version information;
- configuration names;
- target health;
- recent error codes;
- sanitized logs.

It MUST exclude authentication secrets.

22. Security and Privacy Requirements

22.1 Local-Only API

The API MUST bind to localhost by default.

22.2 Credentials

The system MUST NOT collect target passwords.

It MUST NOT store them in:

- database;
- source code;
- logs;

- task definitions.

22.3 Existing Authenticated Sessions

The automation uses the user's already authenticated target sessions.

22.4 Browser Profile Permissions

Automation profile directories SHOULD have Windows access limited to the local user account.

22.5 Database Permissions

SQLite file access SHOULD be limited to the user account running the application.

22.6 File-Path Privacy

File paths can reveal sensitive information. The application SHOULD minimize their display in logs and diagnostic packages.

22.7 Input Validation

All task fields, paths, IDs, configuration values, and API parameters MUST be validated.

22.8 Path Traversal

The application MUST normalize and validate paths and MUST NOT interpret arbitrary user-provided strings as executable commands.

22.9 Command Injection

Task data MUST NEVER be interpolated directly into shell commands.

22.10 Dependency Security

Dependencies SHOULD be pinned and periodically updated.

22.11 Backup Privacy

Backups are as sensitive as the source database and MUST remain local by default.

22.12 Explicitly Prohibited

The system MUST NOT implement:

- CAPTCHA bypass;
- authentication bypass;
- access-control circumvention;

- secret extraction;
 - stealth automation;
 - rate-limit evasion;
 - use of another person's session.
-

23. Configuration Requirements

23.1 Configuration Hierarchy

Priority order:

1. safe built-in defaults;
2. project configuration file;
3. environment variables;
4. database-managed settings;
5. explicit runtime override.

Higher-priority layers **MUST NOT** silently create unsafe settings.

23.2 Configuration Categories

API

- bind host;
- port;
- debug mode.

Database

- database path;
- backup path;
- SQLite journal mode.

Scheduler

- polling interval;
- wake horizon;

- recurrence defaults.

Queue

- worker count;
- claim timeout;
- stale-run threshold.

Browser Targets

- executable paths;
- profile paths;
- timeout values.

Desktop Target

- executable path;
- application timeout;
- UI scale assumptions.

File Handling

- allowed extensions;
- max size;
- max count;
- hash verification policy.

Retry

- max attempts;
- delay;
- exponential backoff;
- jitter.

Logging

- log level;
- retention;

- diagnostic evidence.

Power

- auto-sleep enabled;
- inactivity grace;
- AC requirement;
- wake integration.

Paths

- database;
- logs;
- backups;
- screenshots;
- runtime files.

23.3 Runtime vs Restart Settings

Runtime-safe settings:

- task priorities;
- schedules;
- file allowlist;
- log level;
- retry limits.

Potential restart-required settings:

- API port;
 - database location;
 - packaged runtime paths;
 - core browser runtime configuration.
-

24. Validation Rules

24.1 Task Name

- required;
- trimmed;
- non-empty;
- recommended maximum 200 characters.

24.2 Task Type

Must be one of:

PROMPT, CONTINUE

24.3 Target

Must reference an enabled target.

24.4 Conversation

Must reference a configured, enabled conversation.

24.5 Prompt Text

Prompt Task MUST contain non-empty text.

24.6 Continuation Text

Continue Task MUST contain:

Continue

in the initial release.

24.7 Files

Files:

- MUST exist;
- MUST be accessible;
- MUST match allowed extension;
- MUST be within configured size limits;
- MUST be supported by the selected target.

24.8 Schedule

Must contain:

- valid schedule type;
- valid start date/time;
- valid interval where applicable;
- valid time zone.

24.9 Every-N-Hours

interval_hours > 0

24.10 Cross-Field Rules

- Prompt Task requires prompt text.
- Continue Task uses continuation text and no Prompt payload.
- Scheduled task requires target and conversation.
- Files may be attached only where target capability supports attachment.
- Disabled target cannot receive new enabled schedules.
- Expired one-time task cannot be newly queued.
- A task already running cannot be manually duplicated by another queue entry.

25. Acceptance Criteria

AC-001 — Create Prompt Task

Given a valid target, conversation, prompt, and schedule

When the user creates the task

Then the task is stored as SCHEDULED.

AC-002 — Create Continue Task

Given a valid target and conversation

When the user creates a Continue Task

Then the stored continuation text is Continue.

AC-003 — Task Persistence

Given a scheduled task

When the application restarts

Then the task remains available with its next-run information intact.

AC-004 — Chrome Target

Given a validated Chrome automation profile

When an eligible task runs

Then the Chrome target adapter can open the configured conversation.

AC-005 — Brave Target

The equivalent behavior **MUST** work for Brave.

AC-006 — Edge Target

The equivalent behavior **MUST** work for Edge.

AC-007 — Claude Desktop

A validated Claude Desktop setup **MUST** execute supported Prompt and Continue operations through the DesktopExecutor.

AC-008 — File Path Storage

Given a selected file

When a Prompt Task is saved

Then only the configured file reference and metadata are persisted, not binary content.

AC-009 — File Validation

Given a task referencing a deleted file

When execution begins

Then the task **MUST NOT** send the prompt and **MUST** enter an appropriate manual-action or blocked state.

AC-010 — Queue Ordering

Given multiple due tasks

When the queue selects work

Then only one task is claimed by the single worker at a time and ordering follows the defined priority rules.

AC-011 — Continue Execution

Given a valid Continue Task
When executed
Then the target receives exactly Continue.

AC-012 — One-Time Completion

Given a successful one-time task
When execution completes
Then no future run is created.

AC-013 — Recurring Continue

Given an every-N-hours Continue Task
When one run succeeds
Then the next run is calculated according to the configured recurrence policy.

AC-014 — Retry

Given a retryable transient failure
When the attempt fails
Then the task enters RETRY_SCHEDULED if attempts remain.

AC-015 — Uncertain Send

Given the system cannot determine whether a message was submitted
When execution ends
Then the system MUST NOT blindly resend.

AC-016 — Sleep Protection

Given an execution is active
When automatic sleep logic evaluates system state
Then sleep MUST be suppressed.

AC-017 — Wake

Given a valid wake-capable Windows configuration
When a scheduled wake occurs
Then the agent attempts to process due tasks.

AC-018 — Late Wake

Given the laptop wakes after the intended scheduled time
When the agent resumes
Then the task is processed according to missed-run policy.

AC-019 — Restart Recovery

Given a stale running task from a crashed process

When the agent starts

Then the task is recovered according to execution evidence and uncertain-outcome rules.

AC-020 — Dashboard

Given scheduled and failed tasks

When the Dashboard opens

Then counts, next run, and failure status are visible.

AC-021 — API Binding

Given a default installation

When the backend starts

Then it binds only to localhost.

AC-022 — Database Integrity

Given an invalid foreign-key reference

When persisted

Then the transaction fails.

AC-023 — Backup

Given a valid database

When backup is requested

Then a recoverable local backup is created.

AC-024 — Configuration

Given a changed browser executable path

When the target is revalidated

Then the updated configuration is used without modifying task history.

26. Test Strategy

26.1 Unit Testing

Test:

- task validation;
- state transitions;

- schedule calculations;
- retry calculations;
- queue ordering;
- file validation;
- configuration parsing.

26.2 Database Testing

Test:

- constraints;
- migrations;
- claiming;
- rollback;
- backup;
- restore;
- stale-run recovery.

26.3 API Testing

Mock browser/desktop adapters.

Test:

- CRUD;
- validation;
- error responses;
- state operations;
- pagination;
- filtering.

26.4 Scheduler Testing

Test with simulated time:

- one-time schedule;

- every-N-hours;
- missed runs;
- pause;
- resume;
- expiration;
- clock adjustments.

26.5 Adapter Contract Testing

Every target adapter MUST satisfy the same core contract.

26.6 Playwright Testing

Mock or controlled test pages SHOULD be used for deterministic unit/integration tests.

Real Claude sessions require separate end-to-end tests on the actual user's machine.

26.7 Desktop Automation Testing

Real Windows desktop tests are required because native UI behavior cannot be reliably mocked at the same level as application code.

26.8 Sleep/Wake Testing

Real Windows testing MUST validate:

- sleep;
- wake;
- task startup;
- interactive session;
- late wake;
- failed wake handling;
- return-to-sleep logic.

26.9 Recovery Testing

Test:

- browser crash;

- agent crash;
- database lock;
- network loss;
- missing file;
- expired session;
- renamed conversation;
- UI locator change.

26.10 User Acceptance Testing

The System Owner MUST execute representative real tasks.

26.11 Regression Testing

Target adapter modifications MUST run:

- adapter contract tests;
- regression test suite;
- real target smoke test.

26.12 Compatibility Matrix

Test Area	Chrome	Brave	Edge	Claude Desktop
Launch	Required	Required	Required	Required
Session readiness	Required	Required	Required	Required
Conversation selection	Required	Required	Required	Required
Prompt	Required	Required	Required	Required
Continue	Required	Required	Required	Required
File attachment	Required if target capability supports	Same	Same	If supported
Timeout	Required	Required	Required	Required
Failure detection	Required	Required	Required	Required
Recovery	Required	Required	Required	Required

27. Operational Requirements

27.1 Installation Prerequisites

The machine SHOULD have:

- supported Windows version;
- Python runtime matching the pinned dependency set;
- Git for development;
- supported browsers;
- Claude Desktop where desired;
- sufficient disk space;
- user permission to configure Task Scheduler;
- Internet access for target services.

27.2 First-Time Configuration

Manual user work:

1. Install dependencies.
2. Create project environment.
3. Create automation browser profiles.
4. Manually authenticate supported Claude sessions.
5. Configure executable paths.
6. Configure conversation references.
7. Configure Windows wake behavior.
8. Perform smoke tests.

AI-generated implementation should supply setup scripts and documentation for these steps.

27.3 Browser Login Setup

The user MUST perform legitimate authentication manually in the automation profiles.

The system MUST NOT collect passwords.

27.4 Daily Use

The user should normally:

1. open local UI;
2. create/modify tasks;
3. select target and conversation;
4. select files;
5. schedule;
6. review dashboard;
7. allow the computer to sleep.

27.5 Startup

On startup:

1. initialize application;
2. validate database;
3. load configuration;
4. recover stale states;
5. start scheduler;
6. start agent;
7. recalculate next wake.

27.6 Shutdown

The application SHOULD:

- stop accepting new execution claims;
- allow active tasks to finish if practical;
- persist state;
- close controlled browser/app resources.

27.7 Backup

The user or automated maintenance task SHOULD create local database backups.

27.8 Restore

Restore **MUST** be an explicit administrative action and **MUST** preserve a pre-restore backup.

27.9 Updates

After target applications or browsers are updated:

1. run target health check;
2. run adapter smoke test;
3. review failures before relying on long-running automation.

27.10 AI-Generated Implementation vs Manual User Work

AI-generated implementation **SHOULD handle**

- application architecture;
- Python code;
- FastAPI routes;
- database models;
- migrations;
- Playwright adapters;
- desktop automation adapter;
- scheduler;
- queue engine;
- logging;
- state transitions;
- retry logic;
- tests;
- packaging scripts;
- documentation.

User **MUST handle**

- installing software where required;

- authenticating target accounts;
- creating/approving automation profiles;
- selecting local source files;
- granting Windows permissions;
- configuring hardware/power settings;
- validating real target behavior;
- resolving authentication/session failures;
- approving final behavior.

28. Project Folder Structure

```
project/
├─ backend/
│  │─ api/
│  │─ services/
│  │─ repositories/
│  │─ schemas/
│  │─ models/
│  └─ core/
│
├─ agent/
│  │─ executors/
│  │─ browser/
│  │─ desktop/
│  │─ adapters/
│  │─ queue/
│  │─ power/
│  └─ recovery/
│
├─ frontend/
│  │─ pages/
│  │─ components/
│  │─ styles/
│  └─ assets/
│
├─ database/
│  │─ app.db
│  │─ backups/
│  └─ seed/
│
├─ migrations/
│  └─ versions/
│
├─ config/
│  │─ default.toml
│  └─ local.toml
│
├─ profiles/
│  │─ chrome_claude/
│  │─ brave_claude/
│  └─ edge_claude/
│
├─ logs/
│
├─ evidence/
│
├─ scripts/
│  │─ install/
│  │─ scheduler/
│  │─ backup/
│  └─ diagnostics/
│
```

```
├── tests/
│   ├── unit/
│   ├── database/
│   ├── api/
│   ├── scheduler/
│   ├── adapters/
│   ├── integration/
│   └── windows/
├── docs/
│   ├── SYSTEM_SPECIFICATION.md
│   ├── setup/
│   ├── operations/
│   └── troubleshooting/
├── packaging/
├── .env.example
├── requirements.txt
├── pyproject.toml
├── README.md
└── CHANGELOG.md
```

Directory Purposes

- backend/ — API and domain services.
 - agent/ — automation runtime.
 - frontend/ — local UI.
 - database/ — SQLite database and backups.
 - migrations/ — schema versioning.
 - config/ — validated configuration.
 - profiles/ — automation browser profiles; these are sensitive.
 - logs/ — application logs.
 - evidence/ — safe execution evidence such as screenshots.
 - scripts/ — installation, Windows scheduling, diagnostics, and maintenance.
 - tests/ — automated tests.
 - docs/ — project documentation.
 - packaging/ — release/build artifacts.
-

29. Phase Mapping and Implementation Handoff

Phase 2 — Development Environment Setup

Specification Sections

- 9
- 13
- 23
- 27
- 28

AI Deliverables

- project skeleton;
- requirements file;
- environment setup;
- startup scripts;
- basic FastAPI application;
- basic SQLite initialization;
- documentation.

Manual User Tasks

- install Python;
- install browsers/desktop target;
- create local project directory;
- execute setup scripts;
- confirm environment.

Test Procedure

- start backend;
- open local UI or health endpoint;
- initialize SQLite;

- run basic tests.

Pass/Fail

Pass when the application launches locally and the database initializes without external services.

Handoff Artifacts

- working repository;
 - environment documentation;
 - dependency lock/pin configuration.
-

Phase 3 — Database + Task Queue Engine

Specification Sections

- 6.1
- 6.6
- 6.7
- 11
- 12
- 13

AI Deliverables

- SQLAlchemy models;
- migrations;
- repositories;
- queue engine;
- state machine.

Manual User Tasks

- run migrations;
- verify database file creation;

- execute tests.

Test Procedure

- create task;
- queue task;
- claim task;
- finish task;
- recover stale task.

Pass/Fail

Pass when state and queue operations survive restart and cannot double-claim work.

Phase 4 — Claude Target and Browser Profile Setup

Specification Sections

- 6.3
- 6.4
- 10
- 22
- 27

AI Deliverables

- target registry;
- browser configuration model;
- health-check framework;
- profile validation routines.

Manual User Tasks

- create automation profiles;
- authenticate target sessions;
- configure executable paths;

- configure profile paths;
- validate each target.

Test Procedure

Run target health checks.

Pass/Fail

All configured targets show expected readiness state.

Phase 5 — Browser Automation Engine

Specification Sections

- 6.8
- 10
- 19
- 20
- 21

AI Deliverables

- Playwright adapter layer;
- Chrome adapter;
- Brave adapter;
- Edge adapter;
- safe locator/error handling;
- evidence capture.

Manual User Tasks

- run real-target smoke tests;
- provide screenshots/errors for debugging.

Test Procedure

Send controlled prompt and Continue operations.

Pass/Fail

Correct conversation receives the correct message without using the personal browser profile.

Phase 6 — Claude Desktop Automation

Specification Sections

- 6.9
- 10
- 19
- 20

AI Deliverables

- DesktopExecutor;
- Windows UI Automation integration;
- Claude Desktop target configuration.

Manual User Tasks

- configure application path;
- authenticate;
- test window behavior;
- provide observed UI details when required.

Test Procedure

Execute Prompt and Continue actions.

Pass/Fail

Desktop target performs the intended action and returns reliable completion evidence.

Phase 7 — File Material Pipeline

Specification Sections

- 6.5
- 16
- 22
- 24

AI Deliverables

- local file selector integration;
- metadata validation;
- file-reference persistence;
- attachment pipeline.

Manual User Tasks

- select actual test files;
- move/delete files during failure tests.

Test Procedure

- valid file;
- missing file;
- changed file;
- unsupported extension;
- locked file.

Pass/Fail

File paths are stored without binary duplication and valid files can be attached to a supported target.

Phase 8 — Web UI Development

Specification Sections

- 15
- 14
- 6.1–6.12

AI Deliverables

- dashboard;
- Prompt Task UI;
- Continue Task UI;
- task list;
- target UI;
- history UI;
- settings;
- validation.

Manual User Tasks

- verify usability;
- create real sample tasks;
- report UI defects.

Test Procedure

Use UI to perform the full create/edit/pause/resume/cancel flow.

Pass/Fail

All supported user actions work through the local Web UI.

Phase 9 — Scheduling and Queue Execution

Specification Sections

- 6.6
- 6.7
- 17
- 20

AI Deliverables

- recurrence engine;
- next-run calculator;
- queue scheduling;
- Windows wake-task synchronization logic.

Manual User Tasks

- create short-delay test schedules;
- verify real execution times.

Test Procedure

Test one-time, recurring, every-N-hours, pause, resume, missed execution.

Pass/Fail

Tasks execute according to defined scheduling semantics without duplicates.

Phase 10 — Windows Sleep/Wake Automation

Specification Sections

- 18
- 27

AI Deliverables

- Task Scheduler integration scripts;
- wake-task registration;

- post-execution sleep logic;
- power-event logging.

Manual User Tasks

- configure Windows power settings;
- enable supported wake timers;
- test real sleep/wake behavior;
- verify interactive session conditions.

Test Procedure

Schedule task a few minutes ahead, put laptop to sleep, verify wake and execution.

Pass/Fail

At least one representative Prompt and one Continue task successfully run through the complete sleep/wake workflow under the tested Windows configuration.

Phase 11 — State, Retry, Recovery

Specification Sections

- 12
- 19
- 20
- 21

AI Deliverables

- error taxonomy;
- retry engine;
- stale-state recovery;
- uncertain-outcome handling.

Manual User Tasks

- simulate failures;

- close browsers;
- remove files;
- disconnect network.

Test Procedure

Run failure/recovery matrix.

Pass/Fail

No known failure scenario creates uncontrolled duplicate submission.

Phase 12 — Dashboard, Logs, Monitoring

Specification Sections

- 6.11
- 21
- 22

AI Deliverables

- monitoring screens;
- execution evidence;
- filtering;
- diagnostic bundle generation.

Manual User Tasks

- verify diagnostics;
- review logs;
- verify sensitive data is not exposed.

Test Procedure

Trigger success and failure scenarios.

Pass/Fail

Every major execution state is observable through the dashboard or logs.

Phase 13 — Packaging and Production

Specification Sections

- 6.13
- 23
- 27
- 28

AI Deliverables

- packaged build;
- installation scripts;
- startup/shutdown scripts;
- backup/restore tools;
- final documentation.

Manual User Tasks

- install production build;
- configure target profiles;
- test after reboot;
- test sleep/wake;
- perform final acceptance test.

Test Procedure

Clean-machine or clean-profile installation test.

Pass/Fail

Production package installs, starts, executes representative tasks, survives restart, and preserves data.

30. Requirements Traceability Matrix

Requirement ID	Requirement Summary	Source	Classification	Architecture Component	Database Entity	Related API/UI	Acceptance Criteria	Phase	Status
FR-001	Create tasks	Planning + workflow	Confirmed Requirement	Task Service	tasks	POST /tasks	AC-001	3,8	Defined
FR-011	Prompt Task	Planning + workflow	Confirmed Requirement	Task Service	tasks	Prompt UI	AC-001	3,8	Defined
FR-012	Continue Task	Planning + workflow	Confirmed Requirement	Task Service	tasks	Continue UI	AC-002, AC-011	3,8	Defined
FR-014	Four target registry	Planning baseline	Confirmed Requirement	Target Manager	targets	Targets UI	AC-004–007	4	Defined
FR-020	Conversation reference	Workflow	Derived Requirement	Conversation Service	conversations	Conversation UI	AC-001	4,8	Defined
FR-026	Local file selection	Planning + workflow	Confirmed Requirement	File Service	task_files	Prompt UI	AC-008	7	Defined
FR-033	No binary file storage	Planning baseline	Confirmed Requirement	File Service	task_files	None	AC-008	7	Defined
FR-036	One-time scheduling	Planning baseline	Confirmed Requirement	Scheduler	schedules	Task UI	AC-012	9	Defined
FR-038	Every-N-hours	Planning baseline	Confirmed Requirement	Scheduler	schedules	Continue UI	AC-013	9	Defined

FR-047	Single worker	Architecture decision	Recommended Default	Queue Engine	queue_en tries	Dashboard	AC-010	3,9	Defined
FR-054	Persistent browser profiles	Planning + prior workflow	Confirmed Requirement	Browser Adapter	targets	Targets UI	AC-004–006	4,5	Defined
FR-060	Desktop executor	Planning baseline	Confirmed Requirement	Desktop Executor	targets	Targets UI	AC-007	6	Defined
FR-070	Execution evidence	Derived Requirement	Derived Requirement	Logging	execution_events	Run Details	AC-020	11, 12	Defined
FR-072	Dashboard	Planning	Confirmed Requirement	Frontend	—	Dashboard	AC-020	8,12	Defined
FR-086	Restart recovery	Planning	Derived Requirement	Recovery Service	execution_runs	Health	AC-019	11	Defined
NFR-001	Durable state	Architecture	Derived Requirement	Database	all	—	AC-003	3	Defined
NFR-020	Localhost API	Security	Confirmed Requirement	FastAPI	—	Settings	AC-021	2,8	Defined
NFR-028	Transactional claiming	Architecture	Derived Requirement	Queue	queue_en tries	Dashboard	AC-010	3,9	Defined
NFR-029	UTC timestamps	Specification	Recommended Default	Scheduler/DB	schedules	UI	AC-013	3,9	Defined
FR-047	Single-worker queue	Specification brief	Recommended Default	Queue Engine	queue_en tries	Dashboard	AC-010	3,9	Defined

FR-049	Duplicate prevention	Workflow + safety	Derived Requirement	Queue/Recovery	queue_en tries	Run Detail	AC-015	3,11	Defined
FR-065	Desktop failure handling	Planning	Derived Requirement	Desktop Executor	execution_attempts	Logs	AC-007	6,11	Defined
FR-080	File restrictions	Specification	Recommended Default	File Service	task_files	Settings	AC-009	7	Defined
FR-082	Sleep policy	Planning	Confirmed Requirement	Power Manager	system_settings	Settings	AC-016	10	Defined
FR-083	First launch	Operational requirement	Derived Requirement	Startup Service	schema_migrations	Health	AC-021	2,13	Defined
FR-087	Backup /restore	Operational	Derived Requirement	Database Service	—	Settings	AC-023	13	Defined

31. Assumptions

ASM-001 — Single Primary Windows Computer

Statement

The system initially operates on one Windows computer.

Rationale

The project is explicitly local-first and excludes mandatory cloud infrastructure.

Impact If False

Architecture would require multi-agent coordination.

Validation

Phase 1/2 review.

Owner

System Owner.

Required Validation Phase

Phase 1.

ASM-002 — User Maintains Authentication

The user will manually establish and maintain legitimate authentication in supported target applications.

If false, authentication management becomes a separate project.

ASM-003 — Interactive Session Is Available

GUI automation requires a usable Windows interactive session.

If false, target execution may fail.

Validation: Phase 10.

ASM-004 — Internet Is Available for Target Execution

Target services may require Internet connectivity.

Validation: Phase 5 onward.

ASM-005 — Target Conversations Can Be Identified Visibly

Each desired conversation can be identified through a supported locator mechanism.

If false, task execution cannot safely target the conversation.

Validation: Phase 4/5/6.

ASM-006 — Local Files Remain Reachable

Stored paths refer to files that remain accessible.

Validation: Phase 7.

ASM-007 — Browser Profiles Can Remain Persistent

The chosen automation profiles preserve the necessary session state.

Validation: Phase 4/5.

ASM-008 — Claude Desktop UI Is Automatable

The installed Claude Desktop version exposes sufficient observable Windows UI controls.

Validation: Phase 6.

ASM-009 — Windows Wake Behavior Is Supported

The specific hardware/firmware/Windows configuration permits supported wake scheduling.

Validation: Phase 10.

ASM-010 — The User Accepts Manual Intervention

Some failures cannot be safely recovered automatically and require user action.

Validation: Phase 11.

32. Risks and Mitigations

Risk ID	Description	Cause	Probability	Impact	Affected Component	Mitigation	Contingency	Validation Phase
RSK-001	Target UI changes	External application updates	High	High	Adapters	Locator abstraction	Manual adapter repair	5/6
RSK-002	Locator breakage	DOM/UI changes	High	High	Browser	Fallback locators	Manual action	5

						+ smoke tests		
RS K-003	Session expiry	External auth rules	Medium	High	Targets	Health checks	Manual re-auth	4/11
RS K-004	Locked Windows session	Power/security state	Medium	High	Agent	Detect session state	Manual unlock	10
RS K-005	Missed wake	Hardware/firmware	Medium	High	Scheduler	Catch-up rules	Manual execution	10
RS K-006	Power loss	Battery/outage	Medium	High	System	Persist state	Resume after restart	10/11
RS K-007	File moved/deleted	Local user action	High	Medium	Files	Preflight validation	Manual correction	7
RS K-008	SQLite lock	Concurrent process	Medium	Medium	DB	Single worker + WAL	Retry transaction	3
RS K-009	Duplicate prompt	Uncertain send state	Medium	High	Executor	Uncertain outcome state	Manual verification	11
RS K-010	Response mis-detected	Dynamic target UI	Medium	High	Executor	Completion contract + evidence	Manual action	5/6
RS K-011	Rate limiting	External service policy	Medium	High	Target	Do not evade; retry only when safe	Manual wait	11

RS K- 01 2	Sensitive logs	Poor logging	Medium	High	Logging	Redaction rules	Remove/rotate logs	12
RS K- 01 3	Unsupported file	Target limits	Medium	Medium	Files	Capability checks	Manual change	7
RS K- 01 4	Packaging difference	Runtime/environment mismatch	Medium	Medium	Packaging	Clean-machine test	Manual repair	13
RS K- 01 5	Browser profile collision	Same profile used concurrently	Medium	High	Browser	Dedicated automation profiles	Manual recovery	4/5
RS K- 01 6	Database corruption	Disk failure/interruption	Low	High	Database	Backups + transactional writes	Restore backup	3/13
RS K- 01 7	Task drift	Recurrence semantics	Medium	Medium	Scheduler	Explicit recurrence modes	Manual schedule reset	9

33. Open Decisions Requiring User Approval

DEC-OPEN-001 — Full AI Response Storage

Question

Should full target responses ever be stored by the system?

Why It Matters

Full responses may contain sensitive information and consume substantial disk space.

Options

1. Store only metadata/evidence.
2. Store complete responses.

3. Make response capture task-specific.

Recommended Option

Option 1 for initial release.

Consequences

Maximum privacy and minimum storage.

Default Used

Metadata and evidence only.

DEC-OPEN-002 — Custom Continue Text**Question**

Should the user be allowed to replace Continue with custom text?

Recommended Option

No for initial release.

Consequences

The Continue Task remains semantically distinct and predictable.

Default Used

Fixed Continue.

DEC-OPEN-003 — Auto-Sleep**Question**

Should the system automatically put the laptop back to sleep?

Recommended Option

Yes, configurable and enabled only after safe-state checks.

Consequences

Matches the intended unattended workflow but requires careful user-activity and active-operation detection.

Default Used

Enabled, subject to protected-operation safeguards.

DEC-OPEN-004 — Missed Recurring Runs**Question**

Should every missed recurrence execute immediately after wake?

Recommended Option

No. Skip stale recurring occurrences and execute the next valid occurrence.

Consequences

Prevents bursts of outdated Continue submissions.

Default Used

Skip stale recurring occurrences.

DEC-OPEN-005 — File Limits**Question**

What exact file size/count limits should be enforced?

Recommended Option

Use configurable application defaults listed in NFR-030.

Default Used

10 files, 50 MB per file, 250 MB aggregate.

DEC-OPEN-006 — LAN Access**Question**

Should the local Web UI be accessible from another device?

Recommended Option

No in the initial release.

Consequences

Reduces security complexity.

Default Used

Localhost only.

DEC-OPEN-007 — Task Deletion

Question

Should deletion permanently remove task data or archive it?

Recommended Option

Archive by default; permanent delete only as an administrative action.

Default Used

Soft/archive behavior.

DEC-OPEN-008 — React Migration

Question

Should React be required in the first release?

Recommended Option

No.

Default Used

HTML/CSS/JavaScript initial frontend.

DEC-OPEN-009 — Locked Windows Session

Question

Should the project attempt to automate when Windows is locked?

Recommended Option

No.

Consequences

Some unattended scenarios require an already usable interactive session.

Default Used

Locked session causes manual action/recovery behavior.

34. Decision Register

Decision ID	Decision	Status	Rationale	Alternatives	Consequences
D-001	Local-only architecture	Confirmed	User explicitly rejected Oracle/cloud backend	Cloud hybrid	All scheduling/execution stays local
D-002	SQLite initial database	Confirmed	Single-PC local deployment	MySQL/Maria DB	Simpler installation
D-003	Playwright for browser targets	Confirmed	Browser automation requirement	Selenium	Separate browser profile handling required
D-004	Windows UI Automation for Desktop	Confirmed Direction	Native application	Other GUI tools	Desktop behavior more fragile
D-005	Dedicated automation profiles	Recommended Default	Protect personal profiles	Reuse personal profile	One-time manual setup
D-006	Single execution worker	Recommended Default	Avoid GUI collisions	Parallel workers	Lower throughput, higher safety

D-007	Fixed Continue message	Recommended Default	Clear task semantics	Custom text	Less flexibility
D-008	Local file path only	Confirmed	User explicitly wants local file storage	Database binary storage	Files must remain in place
D-009	Localhost API	Confirmed	Security and local-only use	LAN binding	No remote access
D-010	Archive instead of destructive delete	Recommended Default	Preserve history	Hard delete	More storage
D-011	Skip stale recurring occurrences	Recommended Default	Prevent execution bursts	Catch-up all runs	Some missed recurrence is not replayed
D-012	Manual action for uncertain send	Recommended Default	Prevent duplicate submission	Automatic retry	More manual intervention
D-013	No locked-session automation	Recommended Default	Avoid login bypass/security complexity	Auto-unlock	Requires usable interactive session
D-014	Auto-sleep with safeguards	Confirmed Direction	Matches intended user workflow	Stay awake	Requires power/session checks

35. Definition of Ready for Phase 2

Phase 2 MUST NOT begin until the following are satisfied:

- Phase 1 specification reviewed.
- local-only architecture accepted.
- Task model accepted.
- Prompt Task behavior accepted.
- Continue Task behavior accepted.

- target list accepted.
- conversation reference strategy accepted.
- local file path-only design accepted.
- scheduling semantics accepted.
- queue ordering accepted.
- single-worker policy accepted.
- state machine accepted.
- retry and uncertain-outcome policy accepted.
- Windows wake/sleep limitations understood.
- security boundaries accepted.
- database design accepted.
- API structure accepted.
- UI scope accepted.
- acceptance criteria available.
- risks understood.
- blocking open decisions resolved or explicitly accepted as defaults.
- Phase 2 environment prerequisites identified.

36. Phase 1 Pass/Fail Criteria

Pass Conditions

Phase 1 PASSES only when:

- complete final system scope is documented;
- all major requirements have stable IDs;
- task and execution lifecycles are explicit;
- architecture diagrams are valid;
- database schema is implementation-ready;

- file rules are explicit;
- scheduling semantics are explicit;
- queue behavior is explicit;
- browser and desktop behaviors are distinguished;
- Windows wake/sleep limitations are documented;
- error taxonomy is defined;
- retry and recovery policy is defined;
- acceptance criteria are objective;
- risks and assumptions are visible;
- open decisions are isolated;
- later phases can proceed without redefining core system behavior.

Failure Conditions

Phase 1 FAILS if any of the following remain true:

- task states are contradictory across sections;
 - database cannot represent a required workflow;
 - UI behavior has no backend representation;
 - scheduled tasks cannot survive restart conceptually;
 - retry rules permit uncontrolled duplicate sends;
 - one-time and recurring tasks are indistinguishable;
 - browser and desktop automation are treated as identical;
 - file path-only requirements are violated;
 - architecture depends on Oracle Cloud or another mandatory cloud service;
 - wake/sleep behavior is presented as guaranteed when it is not;
 - requirements are presented as confirmed when they are only assumptions;
 - acceptance criteria cannot objectively determine success.
-

37. Glossary

Term	Definition
Task	Reusable user-defined automation definition
Prompt Task	Task that sends user-defined prompt text
Continue Task	Task that sends the exact default Continue instruction
Schedule	Timing definition associated with a task
Queue Entry	Concrete executable work item created from a due task
Execution Run	One scheduled/manual execution instance
Execution Attempt	One attempt within an Execution Run
Target	Browser or desktop AI application environment
Target Adapter	Adapter isolating target-specific UI behavior
Conversation	User-configured conversation target
Automation Profile	Browser profile dedicated to automation
File Reference	Local path and metadata pointing to a source file
Retry	Re-execution of a failed run under policy
Recovery	Process of returning an interrupted/failed task to a safe state
Wake Timer	Windows scheduling capability used to wake the computer
Local-first	Architecture where all required services operate on the user's computer
Manual Intervention	User action required because automation cannot safely continue
Uncertain Outcome	Condition where the system cannot determine whether an externally visible action completed
Task Definition	Persistent reusable definition of what should be executed
Queue Claim	Atomic reservation of queued work by a worker
Fixed-Rate Recurrence	Next run based on the scheduled series rather than completion time
Fixed-Delay Recurrence	Next run based on completion of the previous successful run
Interactive Session	A usable Windows user session in which GUI automation can operate
Evidence	Safe diagnostic information showing what happened during execution

38. Final Completeness Checklist

- Complete system specification
- Project scope
- Functional requirements
- Non-functional requirements

- Feature list
- Architecture diagrams
- Target model
- Task data model
- Task lifecycle
- State machine
- Database design
- API requirements
- UI requirements
- File-handling rules
- Scheduling rules
- Queue rules
- Browser automation rules
- Desktop automation rules
- Windows wake/sleep rules
- Error-handling rules
- Retry and recovery
- Logging and monitoring
- Security and privacy
- Configuration
- Validation
- Acceptance criteria
- Test strategy
- Operational requirements
- Folder structure
- Phase mapping

- Requirements traceability matrix
 - Assumptions
 - Risks
 - Open decisions
 - Definition of ready
 - Pass/fail criteria
 - Glossary
-

Final Architecture Decision Summary

The final initial-release architecture is:



The system is intentionally local-only. No Oracle Cloud server, VPS, cloud API backend, cloud database, or cloud file-storage service is required for the initial release.

The attached specification brief explicitly requires that later phases implement a self-contained Windows solution with Python, FastAPI, SQLite, Playwright, Windows UI automation, Windows Task Scheduler integration, local filesystem storage, Prompt/Continue tasks, and the defined scheduling/execution lifecycle.